



University of Novi Sad
FACULTY OF TECHNICAL SCIENCES, NOVI SAD



Papp Tamás

Fraktal renderovanje sa hibridnom paralelizacijom

Bachelor Thesis
- Bachelor Academic Studies -

Novi Sad, 2026.

Contents

1. Introduction	1
1.1 Motivation	1
1.2 Goals	1
2. Background	3
2.1 Fractals, General Concepts and Applications	3
2.2 Related Work	4
2.3 The Mandelbrot Set, Definition and Properties	5
2.4 Escape Time Rendering Basics	6
3. Rendering Algorithms	8
3.1 Baseline Scalar Escape Time	8
3.2 SIMD Vectorization	8
3.3 Geometric Early Exit Tests	9
3.4 Mariani-Silver Boundary Subdivision	10
3.5 Distance Estimation Method	11
3.6 Perturbation Theory	12
3.7 Series Approximation	14
3.8 Numerical Precision for Deep Zoom	14
3.9 Traversal and Locality	15
4. GPU Implementation	17
4.1 WebGPU Kernels	17
4.2 CUDA Backend	17
4.3 GPU traversal and memory layout	19
5. Hybrid CPU and GPU Scheduler	21
5.1 Motivation & Design	21
5.2 Tile Decomposition and Classification	22
5.3 Work Distribution & Synchronization	24
5.4 Adaptive Load Balancing with a PI Controller	24
6. Experimental Methodology	26
6.1 Technology Stack	26
6.1.1 Rust	26
6.1.2 GPU Technologies	27
6.1.3 Benchmarking Technology	27
6.2 Hardware	28
6.3 Benchmarks and Test Scenes	29
6.3.1 Fractals, Resolutions, and Iteration Counts	29
6.3.2 Viewports	29
6.3.3 Zoom Sweeps	30

6.3.4	Cross Renderer Comparison	30
6.4	Metrics	30
6.4.1	Frame Time and Throughput	30
6.4.2	Compute Efficiency	31
6.4.3	Precision Controlled Comparison	31
6.4.4	Parallel Scaling and Energy	31
6.4.5	Correctness and Microarchitectural Metrics	31
7.	Results and Discussion	32
7.1	CPU Baseline and SIMD Scaling	32
7.2	GPU Backends	34
7.3	End to End Pipeline	36
7.4	Hybrid CPU+GPU Scheduler	38
7.5	Perturbation Theory	39
7.6	Cross-Project Comparison	40
7.7	Energy Efficiency	42
7.8	Numerical Validation	43
8.	Conclusion	45
	Biography	49

1. INTRODUCTION

1.1 Motivation

Fractals are one of the more interesting corners of mathematics. A shape such as the Mandelbrot set is defined by a short, simple rule, yet applying that rule over and over produces a boundary of unlimited detail [12]. The same kind of structure shows up outside pure math, in coastlines, in heartbeats, in financial markets, and it has practical uses in data compression, network design, and medical imaging.

Rendering a fractal such as the Mandelbrot set relies on the escape time algorithm. For each pixel, a short numerical loop runs until the value either grows past a fixed bound or a maximum iteration count is reached. One pixel's computation does not depend on any other pixel's result, so the whole image can, in principle, be computed in parallel. In practice, most fractal rendering software available to the public still processes pixels one at a time on a single CPU thread, leaving the rest of the CPU's cores and the GPU idle. Software that does use the GPU rarely also uses the CPU at the same time. When it does use the CPU, that is usually only as a fallback for machines without a supported GPU, not as an extra source of compute.

Whether the GPU is actually the faster option is not as settled as it looks. Naive, single threaded CPU code makes GPU speedups look larger than they are, so a fair comparison has to start from a properly optimised CPU baseline [10]. A typical personal computer already has a multicore CPU with SIMD instructions and a GPU sitting next to each other. Using only one of them wastes hardware the user already owns. That is the motivation for this thesis. It sets out to build a renderer that pushes the CPU path and the GPU path each as far as they go on their own. It then combines them with a scheduler that splits the work between CPU and GPU based on which one is actually faster for a given part of the image [11]. That replaces the usual approach of treating "CPU renderer" and "GPU renderer" as two separate programs.

1.2 Goals

The goal of this thesis is to render the Mandelbrot set as fast as possible on a single personal computer, using the CPU and the GPU together rather than either one alone. Reaching that goal means working through several smaller objectives, each covered by a later chapter.

The first is the CPU renderer itself. Starting from a scalar escape time baseline, the thesis adds multicore parallelism, SIMD (f64x4, f32x8), and geometric early exit tests, the cardioid and period 2 and period 3 checks, that skip the iteration loop for pixels already known to lie inside the set. On top of that, the distance estimation method and Mariani–Silver boundary subdivision cut down wasted iteration work by filling large uniform regions of the image without iterating every pixel in them. Tiled traversal ordered by Hilbert or Morton curves further improves memory access locality. A separate concern is deep zooming. Plain double precision arithmetic runs out of precision long before an interesting zoom level is reached. The renderer therefore also needs perturbation theory, using a single reference orbit or multiple reference orbits with rebasing, series approximation, and double-double arithmetic to stay accurate and fast [7].

The second objective is the GPU side. It means implementing the same escape time kernels as GPU compute shaders, once portably with wgpu and WebGPU and once as a hand tuned CUDA backend. Each implementation uses a traversal order and memory layout suited to the GPU rather than porting directly from the CPU code.

The third objective ties the first two together. A hybrid CPU and GPU scheduler splits a frame into tiles, classifies each tile by expected cost, and distributes tiles across CPU worker threads and the GPU at the same time. It then adjusts the split at runtime with a feedback PI controller driven by the measured throughput of each device.

The fourth objective is to measure all of this properly, since none of the above is worth much without evidence. That means fixing a benchmarking setup, the hardware, the benchmark scenes at shallow and deep zoom, and the throughput and latency metrics. It then means measuring what each optimisation actually contributes and comparing the CPU only, GPU only, and hybrid configurations against each other and against existing open source fractal renderers. This last comparison matters on its own. A hybrid scheduler is only worth its added complexity if it measurably beats running the CPU and GPU renderers separately.

2. BACKGROUND

2.1 Fractals, General Concepts and Applications

Before narrowing to the Mandelbrot set specifically, it is worth saying what a fractal actually is in general. A fractal is a shape whose complexity does not go away under magnification. A circle looks more and more like a straight line the closer it is examined, but a fractal keeps showing structure at every scale, and small pieces of it resemble the whole shape, either exactly or in a statistical sense [12]. The Koch curve, the Sierpinski triangle, the Vicsek fractal, and the Sierpinski carpet, shown in Figure 2.1.1, are four of the classic examples. Each is built by repeating a simple replacement rule on a starting shape, over, and over, without end.

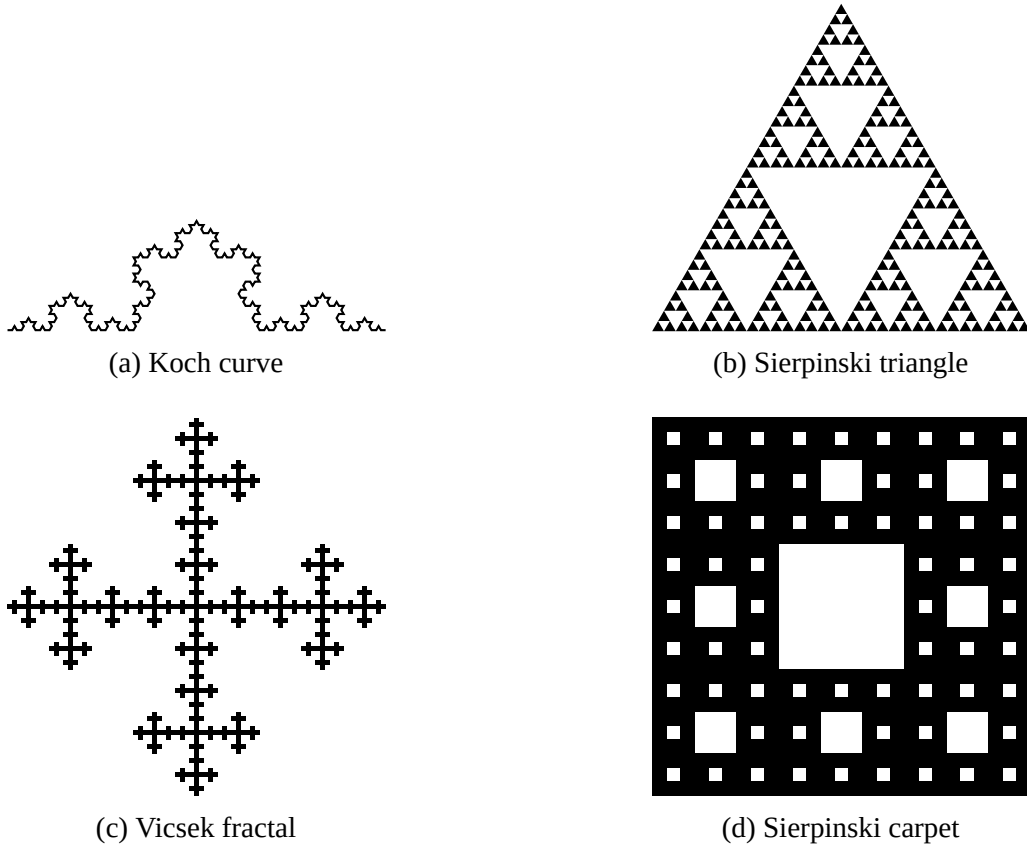


Figure 2.1.1: Four fractals generated by repeating a simple replacement rule on a starting shape. All four are shown after finitely many steps. The true fractal is the limit of this process repeated forever.

That kind of repeated construction gives fractals their most distinctive mathematical property, a dimension that is not a whole number. For a shape built out of N copies of itself, each scaled down by a factor r , the similarity dimension is

$$D = \frac{\log N}{\log(1/r)}. \quad (2.1.1)$$

The Koch curve replaces each segment with $N = 4$ pieces at scale $r = 1/3$, giving $D = \log 4 / \log 3 \approx 1.26$. The Sierpinski triangle uses $N = 3$ pieces at scale $r = 1/2$, giving $D = \log 3 / \log 2 \approx 1.58$. The Vicsek fractal keeps the centre and the four adjacent edge cells of a 3×3 grid ($N = 5, r = 1/3$), giving $D \approx 1.46$, and the Sierpinski carpet keeps the eight outer cells of the same grid and discards the centre ($N = 8, r = 1/3$), giving $D \approx 1.89$. All four values sit strictly between 1 and 2. The shapes are rougher than a line, yet they never thicken into a filled area, and that gap between a shape's fractal dimension and its ordinary topological dimension is, informally, what makes a shape a fractal in the first place [5]. Few naturally occurring fractals are built from an exact repeating rule like these four. Most, and the Mandelbrot set among them, are only self similar in a looser, statistical sense rather than an exact one, a distinction Section 2.3 returns to.

Fractal geometry is not just a mathematical curiosity. Mandelbrot's own starting point was a practical observation: the measured length of a coastline keeps growing the more closely it is measured, because a coastline behaves like a fractal curve rather than a smooth one. Ordinary Euclidean geometry has no good way to describe that [12]. The same kind of branching, self similar structure shows up throughout nature for a functional reason, not by accident. River networks, mountain ranges, and cloud boundaries all have it, and so does the human body, where the lungs and the blood vessels branch fractally to pack a large exchange surface into a small volume. Practical uses follow directly from these observations. Barnsley's iterated function system framework is the same mathematics behind self similar shapes such as the Sierpinski triangle. It was also the basis for an early fractal image compression scheme that stores an image as a small set of transformations rather than as raw pixels [1]. Fractal antenna designs pack a self similar geometry into a small physical footprint to resonate at several frequencies at once, which is why they show up in compact mobile devices. Computer graphics uses fractal noise to generate terrain, clouds, and textures procedurally instead of by hand. Fractal dimension is also used as a diagnostic measurement in medicine, where the irregularity of tissue or blood vessels, quantified as a fractal dimension, can differ measurably between healthy and diseased tissue. The Mandelbrot set, the specific fractal this thesis renders, sits within that same broader family, but it comes from a dynamical system rather than a fixed geometric replacement rule.

2.2 Related Work

Existing fractal renderers fall roughly into three groups, depending on how much of the available hardware they actually use.

The first group renders on the CPU only, and gets its speed from clever algorithms rather than from using more hardware. This group also supplies the four renderers this thesis benchmarks itself against directly in Chapter 7. XaoS [21] is a real time interactive zoomer that stays smooth by reusing pixels computed for the previous frame and only recomputing what changed, rather than recomputing every pixel from scratch on every frame. It now also runs as a web application, but still without using the GPU for the fractal math itself. FractalNow [18] is multi threaded and adds heuristics such as adaptive anti-aliasing, but again keeps all computation on the CPU. FractalRendererCpp [20] is a C++ renderer with interactive mouse and keyboard navigation, and Fractals-rs [3] is a Rust fractal explorer with a SIMD enabled CPU backend. All four are single backend, CPU only renderers with no GPU path, which makes them a useful baseline for how far CPU side optimization alone can go. A fifth CPU only renderer, Fractive [14], is not part of that benchmark set but is worth mentioning on its own. It already vectorizes its CPU kernel with SSE2 on top of multicore

parallelism, exactly the kind of CPU side optimization this thesis also applies. Its use of OpenGL, though, is limited to displaying a 3D height map of the already computed image, not to computing the fractal itself.

The second group targets deep zoom rather than raw speed, and is largely CPU bound by necessity. Kalles Fraktaler and its Nanobrot fork [6] implement perturbation theory. A single high precision "reference" orbit is computed once per frame, and every pixel is then rendered relative to that reference using cheap double precision arithmetic, which is what makes zoom levels far beyond plain double precision practical at all [7]. This numerical technique is orthogonal to parallelization, and Section 3.6 revisits it directly for the renderer built in this thesis.

Outside of fractal rendering specifically, the general problem of splitting a data parallel workload between a CPU and a GPU has its own literature. Lee et al. [10] show that the "GPU is up to 100x faster" claims common at the time were largely an artifact of comparing optimized GPU code against unoptimized, single threaded CPU code. They also show that a properly tuned CPU implementation closes most of that gap. This is the argument used in Section 1.1 for why the CPU side of this thesis needed real optimization work before any CPU versus GPU comparison could be trusted. Qilin [11] addresses the complementary problem of how to split work between CPU and GPU once both are reasonably fast. It profiles a small sample of the workload on each device at runtime and adaptively partitions the rest based on the measured throughput, the same general idea that Chapter 5 applies specifically to fractal image tiles.

None of the renderers surveyed above combine SIMD optimized CPU rendering, a GPU compute backend, and a runtime feedback driven scheduler between the two at the pixel tile level. That combination is what the rest of this thesis builds and evaluates.

2.3 The Mandelbrot Set, Definition, and Properties

For a complex number c , consider the sequence obtained by iterating the quadratic map

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c. \quad (2.3.1)$$

The Mandelbrot set M is the set of all $c \in \mathbb{C}$ for which this sequence stays bounded, that is, it does not tend to infinity as $n \rightarrow \infty$ [12]. Every point of the complex plane is either inside M or outside it, and this simple yes or no test, applied to every pixel of an image, is what actually produces a picture of the set. The details of that test are the subject of Section 2.4.

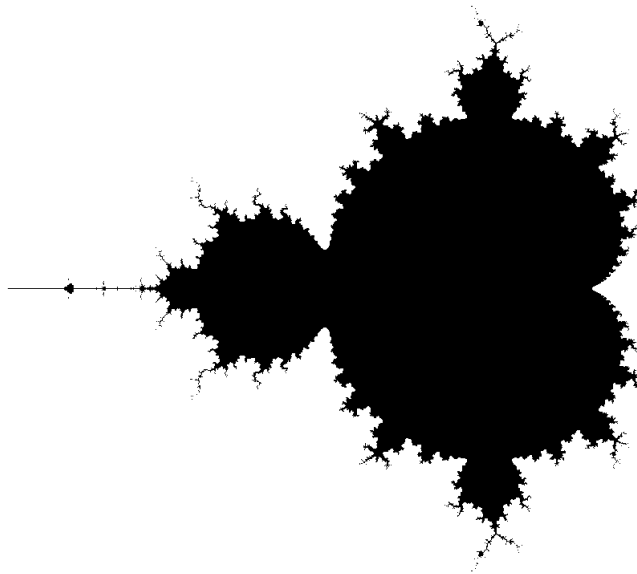


Figure 2.3.1: The Mandelbrot set over $c_r \in [-2.5, 1.0]$, $c_i \in [-1.25, 1.25]$

M has a few properties that matter directly for rendering it, all visible directly in Figure 2.3.1. First, it is bounded. If $|c| > 2$, the sequence in Equation 2.3.1 is guaranteed to diverge, so M lies entirely within the disk $|c| \leq 2$. This follows because once $|z_n| > 2$ and $|z_n| \geq |c|$, each further step at least multiplies the distance from the origin by a factor bigger than 1, so $|z_n|$ grows without bound. The same argument is what justifies stopping the iteration early for any pixel once $|z_n|$ crosses that threshold, instead of iterating forever. Second, M is connected. Despite its visually disconnected looking “islands,” every point of the set can be joined to every other point by a path that stays inside M , a result proved by Douady and Hubbard through the set’s uniformization by external rays. Third, M is self similar in a loose sense. Its boundary contains infinitely many small, distorted copies of the whole set, at every scale, known as satellite or “mini-Mandelbrot” copies. This is precisely why zooming into the boundary keeps revealing new detail rather than converging to a simple curve. It is also the reason deep zoom rendering, Section 3.6, is an interesting problem at all, not just a matter of computing the same picture at a smaller scale. Finally, although M is a two dimensional region of the plane, its boundary is a fractal curve of Hausdorff dimension exactly 2. A curve that space filling is, informally, why the escape time test near the boundary is numerically delicate and why the pixels that take the longest to classify are concentrated exactly where the image has the most visual detail.

2.4 Escape Time Rendering Basics

Equation 2.3.1 cannot be iterated forever on a real computer, so a renderer has to turn the exact yes or no membership test into two practical approximations.

The first is a bailout. Instead of waiting for $|z_n| \rightarrow \infty$, the iteration stops as soon as $|z_n|$ crosses a fixed escape radius. A radius of 2 is already enough to guarantee that the point will diverge, by the same argument used in Section 2.3, so any larger radius is a safe, if sometimes unnecessary, choice. Renderers that want smoother colouring often bail out at a larger radius on purpose, at the cost of a handful of extra iterations per escaping pixel. The second approximation is a maximum iteration count. A pixel that has not escaped once this limit is reached is treated as being inside the set, even though a slightly higher limit might

eventually show it escaping. Raising the limit makes the picture more accurate, particularly near the boundary and at deep zoom, but also makes every pixel that never escapes, typically the majority of the image at high zoom, proportionally more expensive to render. The number of iterations executed is therefore far from uniform across the image. Pixels far from M escape almost immediately, pixels deep inside it run the full iteration limit, and pixels near the boundary can do either, unpredictably. This workload imbalance is the main reason naive per pixel parallelization performs poorly, and it is the underlying problem that the optimizations in Chapter 3 and the scheduler in Chapter 5 both work around.

Turning the raw iteration count into a picture also needs care. Mapping the integer iteration count directly to a colour produces visible banding, sharp rings where the count changes by one, instead of a smooth gradient. The standard fix is smooth, or "normalized," colouring, which uses the escaped value of z_n to compute a fractional iteration count

$$\nu = n + 1 - \frac{\log \log |z_n|}{\log 2}, \quad (2.4.1)$$

so that neighbouring pixels that escaped after, say, 41.3 and 41.7 iterations get visibly different, interpolated colours instead of being lumped into the same band. The range of ν values present in a frame changes completely with zoom level and location. Mapping ν to a colour palette well typically also needs a normalization step, for example histogram equalization. This ensures the available palette entries are spent where the image actually has detail, rather than on iteration counts that barely occur in that particular frame.

Escape time coloring, seahorse valley, before and after normalization

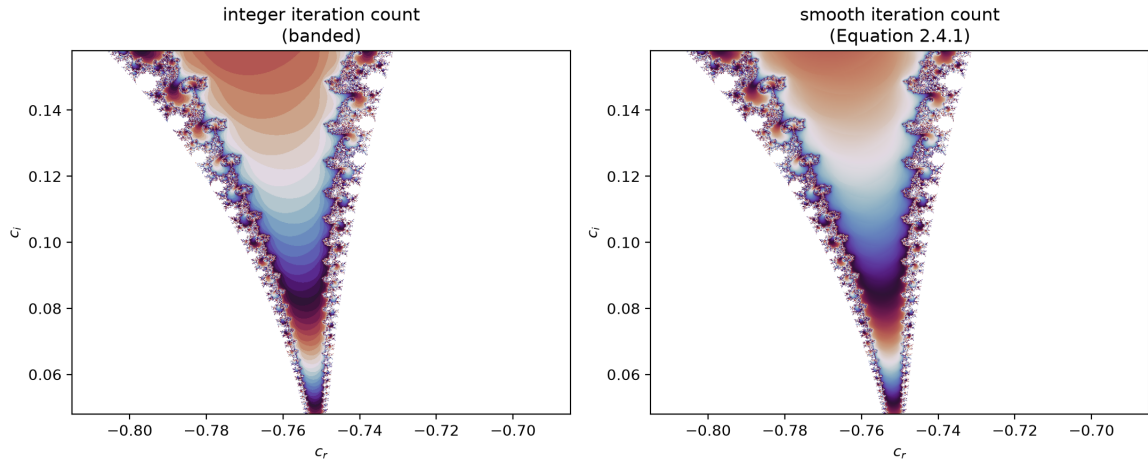


Figure 2.4.1: The same boundary region coloured by raw integer iteration count and by the smooth count from Equation 2.4.1. The left image's rings are the palette repeating once per whole iteration. The right image cycles through the identical palette but interpolates within each ring.

3. RENDERING ALGORITHMS

3.1 Baseline Scalar Escape Time

The renderer’s baseline is a direct, unvectorized implementation of the iteration in Equation 2.3.1, one f64 complex number per pixel, one pixel at a time. Two things separate it from a textbook loop, and both matter for what “baseline” means for the rest of this chapter. First, before iterating at all, it runs the analytic early exit tests from Section 3.3, so the baseline already excludes the two largest interior components of the set from the iteration loop. Every later optimization in this chapter is measured against a baseline that already has this test, not against a naive loop with no shortcuts. Second, the update step $z_{n+1} = z_n^2 + c$ is written as a single fused multiply-add instruction rather than a separate multiply and add, so the hardware does one rounding step instead of two.

The escape test itself uses the radius established in Section 2.4, $|z_n|^2 > 4$, checked as a squared comparison to avoid a square root every iteration. On top of the bailout, the renderer runs Brent style cycle detection [2]. Every few iterations it stores the current z_n , and on later iterations compares against that saved value. If the orbit returns to within 10^{-20} of it, the point is periodic and is immediately classified as being in the set, without waiting for `max_iter`. The comparison interval starts at 8 iterations and doubles after each check, capped at 512. This way the check runs often while it is likely to pay off early and rarely later, once it is unlikely to trigger and would otherwise add per iteration overhead for no benefit. This catches the cycles the cardioid and bulb tests miss. Those tests are closed form for the two largest components of the set. The boundary, though, contains infinitely many smaller periodic components, the visible smaller bulbs and the tiny satellite copies from Section 2.3, that Brent’s method can still catch generically, without knowing their shape in advance.

Work is split across CPU cores in data parallel row chunks, and the pixel loop itself does no per pixel division or trigonometry. The mapping from pixel coordinates to a complex number is reduced, once per frame, to four constants, the real and imaginary start and step, so each pixel’s coordinate is one multiply-add away from the previous one. On the reference machine used throughout this thesis, Chapter 6, this baseline sustains roughly 195 to 239 million pixels per second across resolutions from 800×600 to 3840×2160 for Mandelbrot at 1000 iterations. Throughput is essentially flat with resolution, which is expected for a workload where each pixel’s cost does not depend on the size of the image around it.

3.2 SIMD Vectorization

The scalar loop in Section 3.1 computes one pixel’s iteration at a time even though modern CPUs can apply the same arithmetic operation to several values in a single instruction. The renderer’s SIMD kernels replace the scalar $z_{n+1} = z_n^2 + c$ update with the same update applied to several pixels at once, packed into one vector register, f64x4 with four f64 lanes and f32x8 with eight f32 lanes. Which one runs depends on zoom depth, via the same 10^6 threshold used throughout this thesis to choose between f32 and f64. Below it, the renderer uses the wider, cheaper f32x8 kernel. Above it, f32’s roughly seven decimal digits of precision are no longer enough to tell neighbouring pixels apart, so it falls back to f64x4.

Vectorizing the escape time loop raises a problem the scalar loop does not have. Different lanes escape at different iteration counts, but a SIMD instruction applies the same operation to every lane at once, so no single lane can stop early on its own. The kernels solve this with an active lane mask, a per lane bit pattern of all ones or all zeros, combined with the update via a bitwise AND. Once a lane's $|z_n|^2$ crosses the escape radius, its escape iteration and value of z_n are recorded and the mask clears that lane, so further updates to it become no-ops while the remaining lanes keep going. The loop only stops once every lane has escaped or `max_iter` is reached.

A third kernel, used only for Mandelbrot, pushes further by running two independent `f32x8` chains side by side, sixteen pixels per step rather than eight. The two chains are otherwise identical. The point is instruction level parallelism, not extra vector width. Modern CPUs can have several fused multiply-add instructions in flight at once if a second, independent chain of work is available to fill the pipeline while the first chain's result is still being computed. Measured on 1920×1080 Mandelbrot at 1000 iterations, the two chain kernel renders a frame in 4.25 ms, against 4.62 ms for the single `f32x8` kernel, 6.04 ms for `f64x4`, and 8.17 ms for the scalar baseline, a $1.92\times$ speedup over scalar, of which only about 8% comes from the second chain itself. That the added gain from instruction level parallelism is small is not a surprise once Section 3.3 is taken into account. The cardioid and bulb tests already remove a large share of the interior pixels before the hot loop runs at all, leaving less latency left for a second chain to hide. One criterion run during benchmarking also showed two supposedly identical, repeated measurements of the same kernel disagreeing by more than half. Chapter 6 returns to this as a reminder that a single benchmark run is noise until it has been repeated.

3.3 Geometric Early Exit Tests

Two regions of the Mandelbrot set are large enough, and simple enough in shape, that membership can be tested directly from c without iterating at all, a standard acceleration described in [19]. The main cardioid, the largest component, is the image of the unit circle under $c = \frac{1}{2}e^{i\theta} - \frac{1}{4}e^{2i\theta}$. Equivalently, writing $q = (c_r - 0.25)^2 + c_i^2$, a point lies in the cardioid exactly when

$$q(q + c_r - 0.25) < 0.25 c_i^2. \quad (3.3.1)$$

The period 2 bulb, the large circular component attached to the cardioid's left side, is exactly a disk, $(c_r + 1)^2 + c_i^2 < 0.0625$, of radius 0.25 centred at $c = -1$. Both tests are exact algebraic identities, not approximations, so any point they classify as interior really is interior, and can be assigned `max_iter` directly, with no iteration at all.

The renderer also tests the two period 3 bulbs, the next largest interior components, attached to the cardioid above and below the period 2 bulb. They do not have as clean a closed form as the cardioid, but each is still well approximated by a disk, centred near $c \approx (-0.122561, \pm 0.744862)$ with radius ≈ 0.073715 . Testing squared distance against a squared radius again avoids a square root. Past period 3, the boundary keeps producing smaller bulbs and satellite copies without end, Section 2.3, and it stops being worth hand deriving a closed form test for each one. Smaller periodic components are instead caught generically by the cycle detection already built into the baseline, Section 3.1.

These three tests run first, before the iteration loop, on every pixel in every kernel in this chapter, scalar and SIMD alike. The cardioid and period 2 tests vectorize directly, since they are just arithmetic on the SIMD lanes. The period 3 test, whose disk centres do not vectorize as cleanly, is instead evaluated per lane in scalar precision, only for the pixels the

first two tests did not already resolve. The main cardioid and the period 2 bulb are the two largest regions of the set by area. This test alone therefore accounts for a large share of the pixels that would otherwise run to `max_iter` doing no useful work, exactly the heavy pixels identified as the main source of the workload imbalance in Section 2.4.

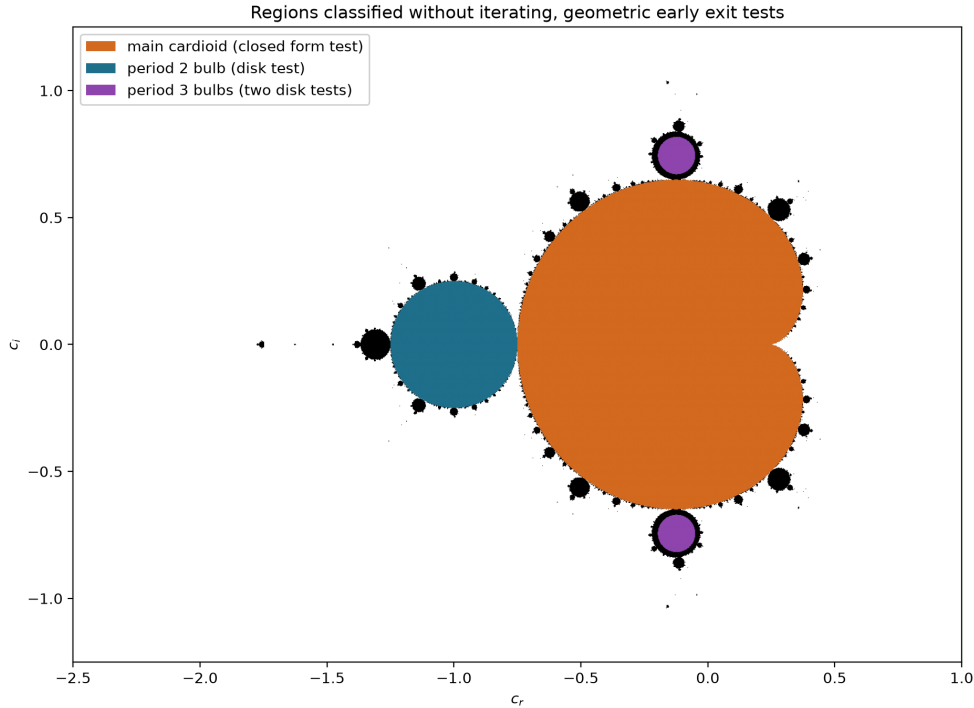


Figure 3.3.1: The three closed form regions over a full frame Mandelbrot render, main cardioid, period 2 bulb, and the two period 3 bulbs. Every pixel inside a shaded region is assigned `max_iter` directly, with no iteration at all. Everywhere else, including the smaller satellite bulbs visible along the boundary, still runs the full loop.

3.4 Mariani-Silver Boundary Subdivision

The tests in Section 3.3 classify individual pixels as interior in closed form. Mariani-Silver subdivision [19] instead exploits the fact that large regions of the *exterior* of the set, far from the boundary, also share the same escape iteration count as their neighbours. The algorithm then tries to detect and fill such regions without computing every pixel inside them.

Given a rectangle of the image, the algorithm renders only its border, the top and bottom rows and the left and right columns. If every border pixel has the same value, the rectangle is assumed uniform and its whole interior is filled with that value without ever touching the pixels inside it. If the border is not uniform, the rectangle is too small to make that assumption safely. It is split in two along its longer axis instead, keeping the two halves closer to square rather than producing long slivers, and the same test is applied recursively to each half. This continues down to a minimum size of 2×2 pixels, below which subdivision no longer pays for itself and the remaining pixels are simply computed directly. Adjacent rectangles share a border, so the frame buffer is seeded in advance with a sentinel value, and a border pixel already computed by a neighbouring rectangle is never recomputed.

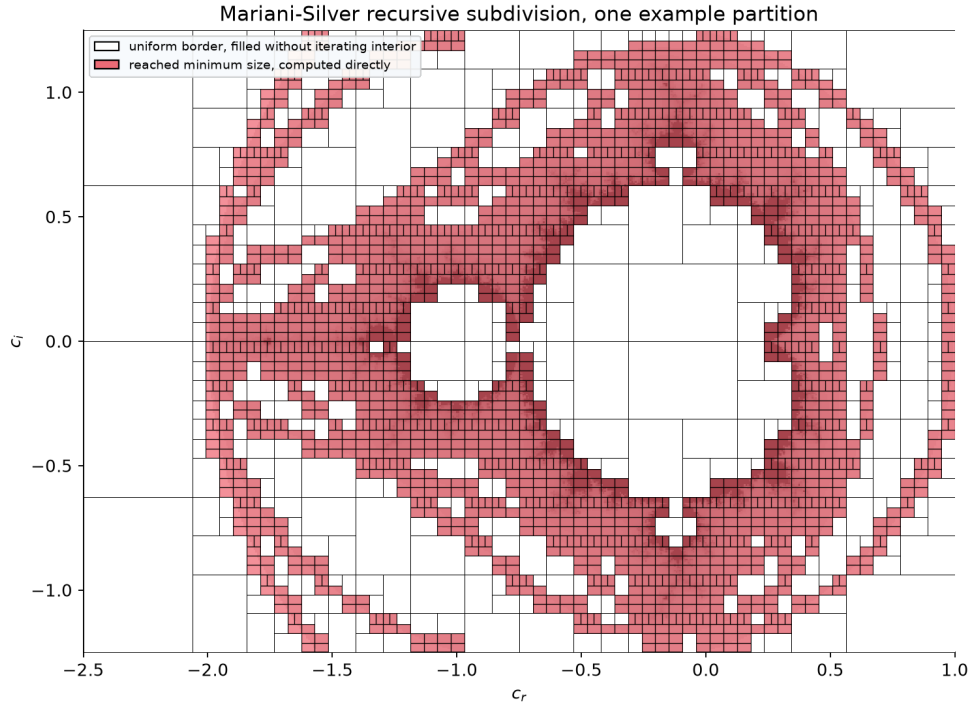


Figure 3.4.1: One example partition over a full frame Mandelbrot render, minimum size raised to 6×6 pixels here for visibility, the renderer itself stops at 2×2 . White rectangles had a uniform border and were filled without touching their interior. Red rectangles reached the minimum size and were computed directly.

The uniform border assumption is not strictly guaranteed. In principle a thin filament of differently valued pixels could thread through a rectangle’s interior without ever touching its border, in which case the fill step would paint over it. In practice this is rare enough, and the visual cost small enough, that the algorithm accepts it as a known, bounded risk approximation rather than guarding against it explicitly.

A second variant folds in the distance estimate from Section 3.5. Instead of only checking whether the border is uniform, it also checks whether every corner of the rectangle is farther from the set’s boundary than the rectangle’s own diagonal, scaled by a tunable safety margin k . If so, the whole rectangle is known to be far enough from any boundary detail that a bilinear interpolation between the four corner values is visually indistinguishable from computing every pixel. The rectangle is filled that way instead, with a sanity check that the corners’ smooth iteration counts do not differ by more than 2.0. This check matters because a bilinear fill across a region where the underlying function changes quickly would visibly misrender it.

3.5 Distance Estimation Method

The distance estimation method [19] answers a different question than the escape time test, not how many iterations until c escapes, but how far c is from the boundary of the set. This quantity’s rigorous justification comes from the potential theory of the set’s exterior [15]. The estimate follows from tracking, alongside the ordinary orbit $z_{n+1} = z_n^2 + c$, its derivative

with respect to c .

$$z_0 = 0, \quad \frac{dz_0}{dc} = 0, \quad z_{n+1} = z_n^2 + c, \quad \frac{dz_{n+1}}{dc} = 2z_n \frac{dz_n}{dc} + 1, \quad (3.5.1)$$

obtained by differentiating the iteration itself term by term. If the orbit escapes at iteration n , the distance from c to the nearest point of the boundary is approximately

$$d \approx \frac{|z_n| \ln |z_n|}{|dz_n/dc|}. \quad (3.5.2)$$

This is a standard result for the exterior distance estimate of an escape time fractal, not specific to this renderer. What is specific to this renderer is how it gets used. Rather than driving a dedicated anti-aliasing or outline rendering mode, d is used exactly once, as the cull test inside the DEM augmented Mariani-Silver variant from Section 3.4. A rectangle whose four corners are all farther from the boundary than their own diagonal is guaranteed not to contain any fine boundary detail, and can safely be filled by interpolation instead of iterated pixel by pixel. Because dz_n/dc is accumulated with one extra complex multiply-add per iteration on top of the ordinary orbit, the distance estimate is nearly free to compute alongside an escape time run that was going to happen anyway. The actual saving comes from how much pixel by pixel work the culling test lets Mariani-Silver skip, not from the estimate itself being cheap in isolation.

3.6 Perturbation Theory

Every kernel described so far iterates c directly in f64 or f32, which stops working once the zoom is deep enough that neighbouring pixels' c values differ by less than a double precision float can resolve. That limit is reached long before the deep zoom regime that Section 2.3's self similar boundary makes interesting. Perturbation theory [13] sidesteps this by iterating a *difference* instead of an absolute value. A single high precision reference orbit Z_n is computed once, for one reference point near the centre of the view. Every other pixel $c = c_{\text{ref}} + \delta c$ is then rendered by iterating the much smaller quantity $\delta_n = z_n - Z_n$ directly.

$$\delta_{n+1} = 2Z_n\delta_n + \delta_n^2 + \delta c, \quad (3.6.1)$$

obtained by subtracting the reference orbit's own recurrence from the pixel's. Because δ_n stays small for most pixels near the reference point, it can be iterated in ordinary f64 even where the corresponding z_n itself would have needed far more precision to represent. This means the expensive high precision work Section 3.8 is paid for once per frame instead of once per pixel.

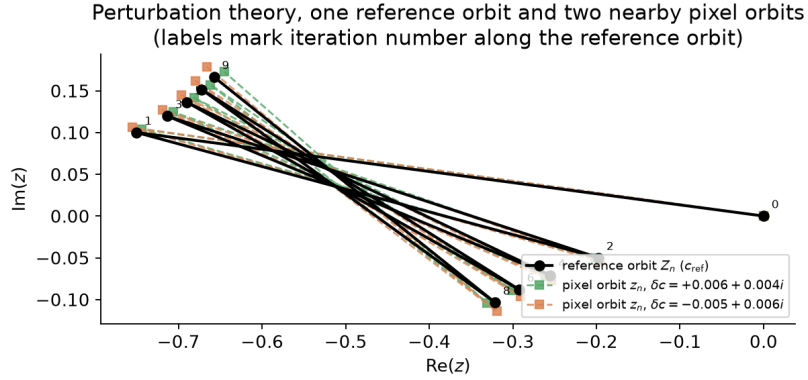


Figure 3.6.1: A real reference orbit at $c_{\text{ref}} = -0.75 + 0.1i$, the seahorse valley point Section 6.3.2 also uses, alongside two pixel orbits a small δc away. All three orbits stay close together for the iterations shown, which is exactly what lets δ_n be represented in ordinary f64 while Z_n itself carries the extended precision.

This breaks down for pixels whose orbit drifts too far from the reference. Once $|\delta_n|$ grows to a non negligible fraction of $|Z_n|$, the approximation the method relies on stops holding, an event usually called a *glitch*. The renderer detects this by comparing $|\delta_n|^2$ against a small fixed fraction of $|Z_n|^2$ and flags the pixel once it is crossed. The plain variant simply falls back to the exact scalar kernel from Section 3.1 for any flagged pixel, which is always correct but pays the full per pixel cost again for exactly the pixels perturbation was meant to avoid computing directly. *Rebasing* [7] avoids most of that fallback without a second reference orbit. Whenever a pixel’s orbit happens to pass close to the origin again, once $|z_n|^2$ drops back below $|\delta_n|^2$, or the stored reference orbit runs out, the current z_n is simply adopted as a new, local delta of 0, and iteration continues against the same reference orbit from that point on, reusing one precomputed orbit as though a fresh reference had been recomputed there for free. *Multi-reference* rendering goes further still. It renders the whole frame against one reference, marks every glitched pixel, and then computes a new reference centered on the first pixel still glitched, since glitches cluster spatially as nearby pixels tend to drift away from a given reference together, repeating up to 8 reference orbits before giving up and falling back to the exact kernel for whatever is still unresolved.

Measured directly, perturbation rendering is not a throughput win. Across the zoom range tested, it renders slower than the direct kernel it replaces, because that range never reaches the depth where f64 actually breaks down, so the perturbation machinery is pure overhead on top of arithmetic the direct kernel could already do unaided.

3.7 Series Approximation

Perturbation iteration from Section 3.6 still runs one δ_n update per pixel per iteration. Series approximation [13, 7] tries to skip the first several hundred iterations of that work entirely, for pixels close enough to the reference orbit that their early behaviour is predictable in closed form.

Substituting a power series guess $\delta_n \approx A_n \delta c + B_n \delta c^2 + C_n \delta c^3 + D_n \delta c^4$ into the perturbation recurrence Equation 3.6.1 and matching powers of δc gives a recurrence for the coefficients themselves, computed once alongside the reference orbit.

$$A_{n+1} = 2Z_n A_n + 1, \quad B_{n+1} = 2Z_n B_n + A_n^2, \quad (3.7.1)$$

$$C_{n+1} = 2Z_n C_n + 2A_n B_n, \quad D_{n+1} = 2Z_n D_n + (2A_n C_n + B_n^2). \quad (3.7.2)$$

As long as the third and fourth order terms stay negligible relative to the leading term for the worst case δc in the frame, the viewport corner farthest from the reference point, the series is a valid stand in for direct iteration. The renderer tracks the last iteration at which that holds, evaluates the series directly at that iteration for every pixel, and then continues ordinary perturbation iteration from there instead of from iteration zero.

On the benchmark scene used for perturbation above, the skippable prefix turns out to be short. It runs to a few tens of iterations out of a thousand, a one to three percent head start rather than the large upfront skip series approximation is capable of at greater depth.

3.8 Numerical Precision for Deep Zoom

Perturbation theory, Section 3.6, moves the precision problem rather than eliminating it. Pixel deltas can stay in f64, but the single reference orbit they are computed against still has to track an absolute position in the complex plane, and at extreme zoom that position needs more than f64's roughly 53 bits of mantissa to distinguish from its neighbours. Rather than pull in an arbitrary precision arithmetic library, the renderer represents an extended precision real number the way Dekker's classic double-double technique does [4], as an unevaluated pair of ordinary f64 values, (hi, lo) , with the invariant that $|lo|$ is at most half a unit in the last place of hi . The true value is the exact sum $hi + lo$, which is never actually computed as a single float. This "double-double" representation gives roughly 106 bits of mantissa, twice f64's, while every individual operation on it still runs as ordinary hardware floating point arithmetic.

The two building blocks are Knuth's TwoSum [9], a short branch free sequence that computes $a + b$ together with its exact rounding error in a second float, and a TwoProduct [17] that gets the same exact error decomposition for a multiplication directly from a fused multiply-add. It computes $p = a \times b$ and then the error term as $a \times b - p$ in a single fused multiply-add instruction, with no double width intermediate needed. Addition on the pair type sums the high parts with TwoSum, folds both low parts into the resulting error term, and renormalizes with a second TwoSum. Multiplication computes the exact high part product with TwoProduct, adds the cross terms $hi_a \cdot lo_b$ and $lo_a \cdot hi_b$ as ordinary f64 products, already small enough not to need their own error term, and renormalizes the same way.

This extended precision is used in exactly one place, computing the reference orbit itself, once the requested zoom passes a fixed threshold, 10^{12} , beyond which plain f64 is no longer trustworthy for that orbit. Every per pixel δ_n update still runs in ordinary f64 against that orbit, truncated back down from the pair representation once it is stored, since the deltas themselves never needed the extra precision, only the shared reference did. Be-

cause the reference orbit is computed once per frame rather than once per pixel, the roughly twofold arithmetic cost of double-double operations over plain f64 stays negligible against total frame time even when it is active.

3.9 Traversal and Locality

Every algorithm so far decides *what* to compute for a pixel. This section is about the order *in which* pixels are visited, and about avoiding recomputation altogether when the previous frame already contains most of the answer.

The renderer divides the frame into square tiles of 64×64 pixels and distributes bands of tiles across CPU worker threads. This fixed tile size is also the unit of work the hybrid scheduler in Chapter 5 dispatches to the CPU and the GPU. Within a tile, pixels can be visited in plain row major order, or along a Hilbert curve [8], a recursively defined path that always steps to a geometrically adjacent pixel and, unlike row major order, never jumps back across the full width of the tile at the end of a row. The Hilbert visiting order for a tile is generated once by interleaving bits and rotating quadrants, and reused unchanged for every tile in the frame, so the per pixel cost is only a table lookup, not a recomputation of the curve.

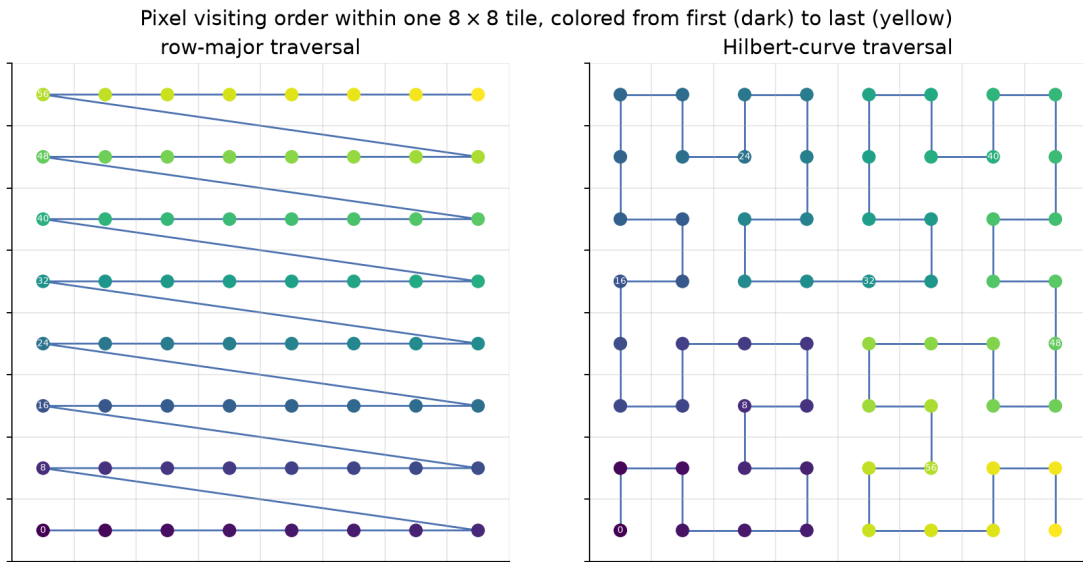


Figure 3.9.1: Pixel visiting order within one tile, row major against Hilbert. Row major jumps back across the full tile width at the end of every row. The Hilbert curve never takes a step that is not to a geometrically adjacent cell.

The motivation for trying a Hilbert order at all is cache locality. Consecutive pixels along the curve are also spatially close in the image, and if a full frame buffer does not fit in cache, that should mean fewer evictions than a row major sweep that revisits a row's neighbourhood only after scanning every other row in between. On this thesis's benchmark hardware, that expected win does not appear. Hilbert traversal renders measurably slower than plain row major order at every resolution tested, and its last level cache miss rate is higher, not lower. The most likely explanation, examined in Chapter 7, is that the frame buffer at the tested resolutions already fits comfortably in the CPU's last level cache with hardware prefetching doing its job on the row major access pattern, so the Hilbert curve's per pixel bit interleaving becomes pure added cost with no locality problem left for it to solve. A different traversal order, Morton or Z-order indexing [16], is used elsewhere in this renderer,

but on the GPU rather than the CPU tiling path described here. Section 4.3 covers how it keeps threads within a GPU warp accessing nearby memory.

Panning the view does not need to touch the traversal order at all, only the amount of work done. When the view shifts along a single axis, most of the previous frame's pixels reappear at a shifted position rather than needing to be recomputed. The renderer copies the reusable region of the old buffer into its new position with a single bulk memory copy and only computes the newly exposed strip along the edge the view moved toward. This turns a full frame recompute into a copy plus a recompute proportional to the pan distance rather than the frame area, but only for panning aligned to one axis. A simultaneous horizontal and vertical shift would need the reused and newly exposed regions expressed as an overlapping copy that a single bulk copy cannot represent, so diagonal panning falls back to computing the frame directly rather than attempting a partial reuse that would not save the intended work anyway.

4. GPU IMPLEMENTATION

4.1 WebGPU Kernels

WebGPU is the portable one of the renderer’s two GPU backends. The same compiled shader runs on any GPU with a Vulkan, Metal, or DirectX driver, at the cost of one hard restriction. WGSL, the shader language WebGPU compiles, has no `f64` type at all, so every kernel in this backend is `f32` only. Unlike the CPU and CUDA backends, this backend has no deep zoom fallback of its own. It stays inside the same 10^6 zoom threshold used throughout this thesis for the `f32` fast path, and anything beyond that threshold renders on a different backend entirely.

Within that limit, the WebGPU kernel mirrors the CPU’s own `f32` kernels closely. It runs the same cardioid, period 2, and period 3 tests from Section 3.3, inlined directly into the shader, and it skips periodicity detection the same way the CPU’s SIMD kernels do, on the same reasoning that Brent’s cycle check is not worth the extra branching at `f32` granularity. The Mandelbrot kernel shares its shader file with the renderer’s other fractal types, selected at runtime by an integer field in a small uniform buffer of view and rendering parameters uploaded once per frame. Two entry points share this kernel code. One dispatches one thread per pixel over the whole frame in row major order, in workgroups of 16×16 . The other instead reads an uploaded array of tile descriptors and uses each workgroup’s position in the dispatch to select one, bounds checking each thread’s local position against that tile’s own width and height before writing to its true position in the frame buffer. This second entry point is what the heterogeneous scheduler’s GPU worker dispatches through (Chapter 5), since it needs to render an arbitrary, scheduler chosen batch of tiles in one call rather than the whole frame.

A separate shader handles perturbation rendering. It uploads the reference orbit (Section 3.6) as two `f32` arrays for the real and imaginary parts, together with its center, and iterates the same delta recurrence, Equation 3.6.1, that the CPU kernels do, falling back to a direct `f32` Mandelbrot evaluation on a glitch or once the uploaded orbit runs out. It does not rebase and does not retry with a second reference. Unlike the CPU path’s three variants, WebGPU perturbation uses a single reference only. Rebasing and multiple references would add control flow that needs to run identically across every thread in a workgroup to stay cheap on a GPU, and this was not judged worth it for a backend whose reach is already capped by `f32`.

Getting the result back to the host costs more than computing it. The kernel writes into a storage buffer on the GPU side. A copy from device to device then moves it into a second buffer flagged for host mapping, which is mapped asynchronously and waited on before its contents are copied into a freshly allocated host buffer. At 1920×1080 , zoom 1, `max_iter = 1000`, the kernel itself runs in 0.29 ms, but the full frame takes 2.26 ms. Readback is roughly 90% of it.

4.2 CUDA Backend

The CUDA backend targets the discrete NVIDIA GPU directly instead of going through a portability layer. The kernel is compiled ahead of time with `nvcc`, targeting the `sm_86` architecture that matches the Ampere generation RTX 3050 this thesis benchmarks on, and the

resulting PTX is embedded into the compiled binary, so nothing needs compiling at runtime. Two of the compiler flags are worth calling out on their own. One keeps denormalized floats alive, `--ftz=false`, since the smooth coloring formula from Section 2.4 needs them for accuracy near the escape threshold. The other turns fused multiply add contraction off entirely, `--fmad=false`. That second choice is the mirror image of Section 3.1’s CPU kernel, which turns FMA *on* for speed and accepts a difference of one rounding step from a separate multiply and add. Here it is turned off so that CUDA’s arithmetic rounds exactly the way the CPU’s does, trading a small amount of GPU throughput for images that are bit identical across backends, which matters when the two are compared directly (Chapter 7).

Unlike the WebGPU backend, this backend keeps its default kernels in `f64`, giving it full parity with the CPU baseline. It runs the same cardioid, period 2, and period 3 tests, and the same Brent style cycle detection from Section 3.1. Below the same 10^6 zoom threshold used everywhere else in this thesis, dedicated `f32` kernels take over instead, mirroring the CPU’s own precision dispatch. A separate kernel handles perturbation, keeping the reference orbit and every per pixel delta in `f64`, unlike the WebGPU backend’s `f32` only path. It does not even receive the reference point’s coordinates as an argument. Since the orbit is always computed for the frame’s own center pixel, the kernel reconstructs c_{ref} directly from the frame’s own geometry. Like the WebGPU backend, it has no rebasing and no retry with a second reference, and neither backend uses the double-double arithmetic from Section 3.8. Zoom levels deep enough to need it still render on the CPU only, in this implementation.

Getting those flags and that precision policy wrong is not a hypothetical risk. The `f32` Mandelbrot kernel’s period 3 test used to call the `f64` version of `in_period3_bulb`, on the reasoning that the CPU’s own `f32` kernel calls its `f64` bulb check the same way, and that check only runs once per pixel rather than once per iteration, so it should not matter. That reasoning holds on a CPU, where `f64` and `f32` arithmetic cost about the same. It does not hold on this GPU. Ampere runs `f64` at roughly $1/64$ the throughput of `f32`, so eight `f64` operations running once per pixel cost as much issue bandwidth as the entire thousand iteration `f32` loop they precede. An ablation that timed the kernel alone, with no host transfer, found the `f64` check version running at 0.910 ms against 0.452 ms once it was replaced with a true `f32` bulb test, a factor of two, confirmed by inspecting the compiled PTX directly. Eight `f64` instructions survived in the old kernel and zero in the new one. The fix was verified bit identical against the original, not just visually similar, by rendering seven viewports chosen to stress the boundary the check actually guards, including views centered directly on the period 3 arc at zoom levels up to the 10^6 precision threshold, and comparing every one of 2,073,600 pixels. Zero differed, in every view. The realised effect on a full rendered frame, including the host transfer that dominates it, was a 15 to 19% reduction across every resolution tested, from 800×600 up to 3840×2160 , and the same fix, being in shared kernel code, applied to the scheduler’s tiled GPU dispatch for free.

As with the WebGPU backend, the transfer back to the host costs more than the kernel does. At the same 1920×1080 configuration, the kernel takes 0.28 ms and the host readback takes 1.33 ms, about 82% of a 1.61 ms total frame. Registering the host destination buffer as page locked memory, via the CUDA driver call `cuMemHostRegister`, lets the transfer DMA directly into caller owned memory instead of first staging through a driver internal pinned buffer the way an unpinned destination would force it to. If that registration is refused, the code falls back to ordinary memory rather than failing outright. This is, in the end, the explanation for why CUDA’s 1.61 ms frame beats the WebGPU backend’s 2.36 ms one. Once the fix above is in place, the two backends’ kernels are within 5% of each other, and essentially all of the remaining gap is that backend’s extra staging copy.

CUDA’s speedup over the CPU depends heavily on which CPU number it is measured against. Against the scalar f64 baseline from Section 3.1, CUDA renders this thesis’s benchmark scene roughly $5.4\times$ faster. Against the CPU’s own best kernel, the two chain f32x8 SIMD kernel from Section 3.2, that margin drops to $2.64\times$, and once a `colorize()` step run on the CPU for both sides is added to the timing, it drops again to $2.09\times$. The GPU kernel and the hardware are identical in all three numbers. Only the CPU side of the comparison changes. That $2.09\times$ figure predates CUDA’s own `colorize` step moving on device (Section 4.3). Section 7.3 reopens the comparison with that change applied. That is the point Section 1.1 made from the literature [10], now measured directly on this thesis’s own renderer. An unoptimized CPU baseline makes a GPU look several times faster than a properly optimized one does.

4.3 GPU traversal and memory layout

The CUDA backend dispatches its full frame kernels in Morton order, also called Z order, rather than row major order [16]. Each thread’s flat index is decoded back into an (x, y) pixel position by interleaving the bits of two separate 16 bit numbers into one 32 bit code, so that indices close together in the launch correspond to pixels close together in the image on both axes at once, rather than only along one row. The launch grid itself is padded to the smallest enclosing power of two square around the frame’s larger dimension, then covered with 16×16 thread blocks. A thread whose decoded position falls outside the true frame simply returns without writing anything. The stated reasoning for dispatching this way, rather than row major, is that escape time cost is spatially correlated (Section 2.4), so keeping the threads in a warp spatially close should keep their iteration counts more uniform, and reduce how long a warp’s fast finishing threads sit idle waiting on its slowest one.

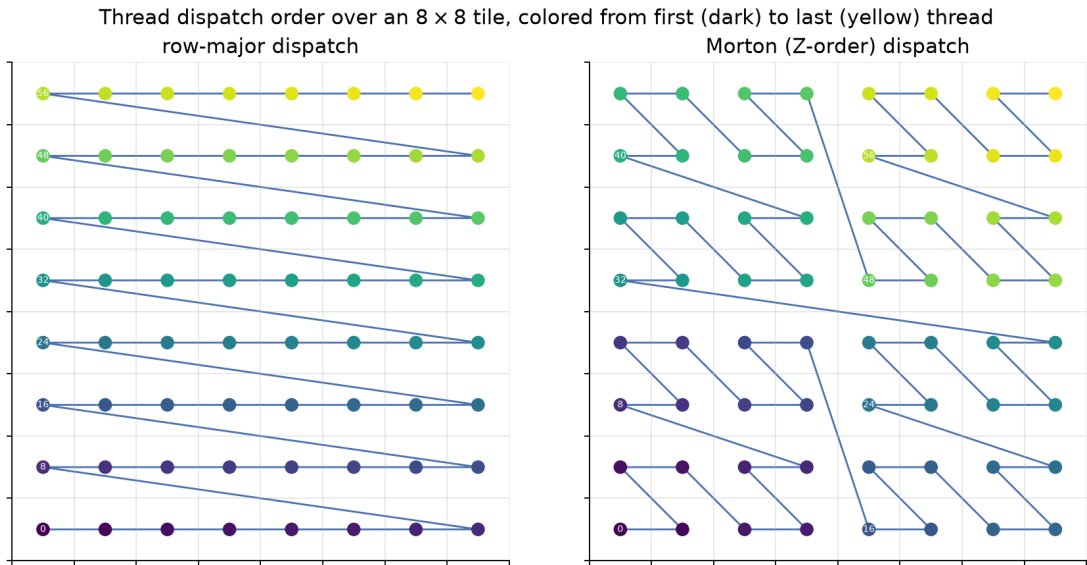


Figure 4.3.1: Thread dispatch order over an 8×8 tile. Row major order jumps back across the full tile width at the end of every row, so consecutive thread indices land far apart in y at those boundaries. Morton order keeps consecutive indices inside the same 2×2 or 4×4 quadrant far more often, the locality property Section 4.3 argues should help a warp’s iteration counts stay uniform.

Measured directly, that reasoning does not pay off on this hardware. A same process, paired comparison against a plain row major grid, using the same kernel math dispatched through the scheduler's tiled entry point instead, shows the Morton launch issuing just over twice as many threads as there are pixels, 4,194,304 against 2,073,600 for a 1920×1080 frame, since padding to a square roughly doubles the grid. The median cost across seven repeated runs was about +1.9%, with two of the seven runs actually favoring the Morton launch. That is indistinguishable from noise. The excess threads land in one contiguous region past the frame's true edge, so whole blocks retire immediately rather than doing any wasted per pixel work, which is why doubling the launched thread count costs next to nothing either way. The padding is nearly free, but so, on this measurement, is the ordering it was meant to enable. This is the same verdict Section 3.9 reached testing a Hilbert curve on the CPU, a space filling traversal order chosen on locality grounds, tested directly against the plain alternative, and found not to move the number on the hardware actually measured.

The heterogeneous scheduler does not use the Morton path at all for the tiles it sends to the GPU. It dispatches through a tiled kernel instead, one block per tile selected by the block's z index, each thread bounds checked against its own tile's width and height. A second variant of that tiled kernel additionally carries a destination offset per tile, for writing into one densely packed output buffer rather than each tile's own position in a full frame, used when the scheduler hands the GPU a scattered subset of the image rather than a contiguous region of it (Chapter 5).

Underneath either traversal order, both backends write into the same kind of buffer, a flat, row major array of one `f32` per pixel holding the raw, possibly fractional escape time value from Equation 2.4.1, not a color, at 4 bytes per pixel. That is 8.29 MB for the 1920×1080 frame measured throughout this chapter. Turning that into a displayed image, histogram, cumulative distribution, and palette lookup, is a separate pass over the same buffer, run as its own pair of kernels on the CUDA side, mirroring the two stage render then colorize split the CPU pipeline already uses.

Sections 4.1 and 4.2 both measured host readback at 80 to 90% of total frame time, an order of magnitude more than the traversal choices in this section moved the number by. The hybrid scheduler in Chapter 5 is built around that readback cost.

5. HYBRID CPU AND GPU SCHEDULER

5.1 Motivation & Design

Chapter 4 ended on a single number. Host readback is 80 to 90% of a GPU frame, on both backends, and no kernel level optimization touches that share. Nothing in this thesis's rendering algorithms shrinks that cost, because it has nothing to do with the algorithm. It is simply the price of moving a finished buffer across PCIe. What a per pixel algorithm *can* change is which device computes that buffer's pixels in the first place, and the two devices are not equally good at every pixel. GPU threads execute in warps of 32, in lockstep, so if one thread in a warp reaches the iteration cap while another escapes after ten iterations, the whole warp keeps running until its slowest member finishes. That is the same masked lane cost the SIMD kernels in Section 3.2 pay at a width of eight, except a stalled GPU warp cannot be handed other work mid flight the way an idle CPU core can. A CPU core has no such constraint. Each of its threads runs its own pixels independently, so a chaotic, high iteration count region costs that core time proportional to its own pixels and nothing more. Regions of the image near the set's boundary, where neighbouring pixels' escape counts diverge sharply, are therefore cheap on a CPU and comparatively expensive on a GPU. Uniform regions, deep in the set or far outside it, are cheap on both, because a GPU's lockstep execution has no divergence penalty left to pay there. The scheduler in this chapter exploits exactly that asymmetry. It routes each part of the image to whichever device pays less for *that* part, in the spirit of adaptive heterogeneous mapping across CPU and GPU [11], rather than rendering the whole frame on one device or splitting it by a fixed rule that ignores content.

The design has four moving pieces. First, before any pixel is rendered, the frame is decomposed into tiles and each tile is classified as suitable for the GPU or suitable for the CPU, cheaply, by sampling only its four corners (Section 5.2). Second, that classification is turned into actual work. GPU classified tiles are dispatched to the GPU while a pool of CPU workers concurrently drains the CPU classified tiles from a shared queue, the two running concurrently for the whole frame rather than one waiting on the other (Section 5.3). Third, a split decided before either device has rendered a single pixel cannot know either device's load in real time, so a fraction of the GPU's tiles are held back as work any idle CPU core can steal once its own queue is empty, correcting for a temporarily slow GPU within the same frame (Section 5.3). Fourth, correcting only within a frame is not enough if one device is *consistently* faster or slower than the classifier's static threshold assumes, since interactive rendering renders the frame again on every pan and zoom. Each frame's own measured GPU and CPU time therefore feeds a PI controller that retunes the classifier's threshold for the next frame, closing the loop across frames rather than within one (Section 5.4).

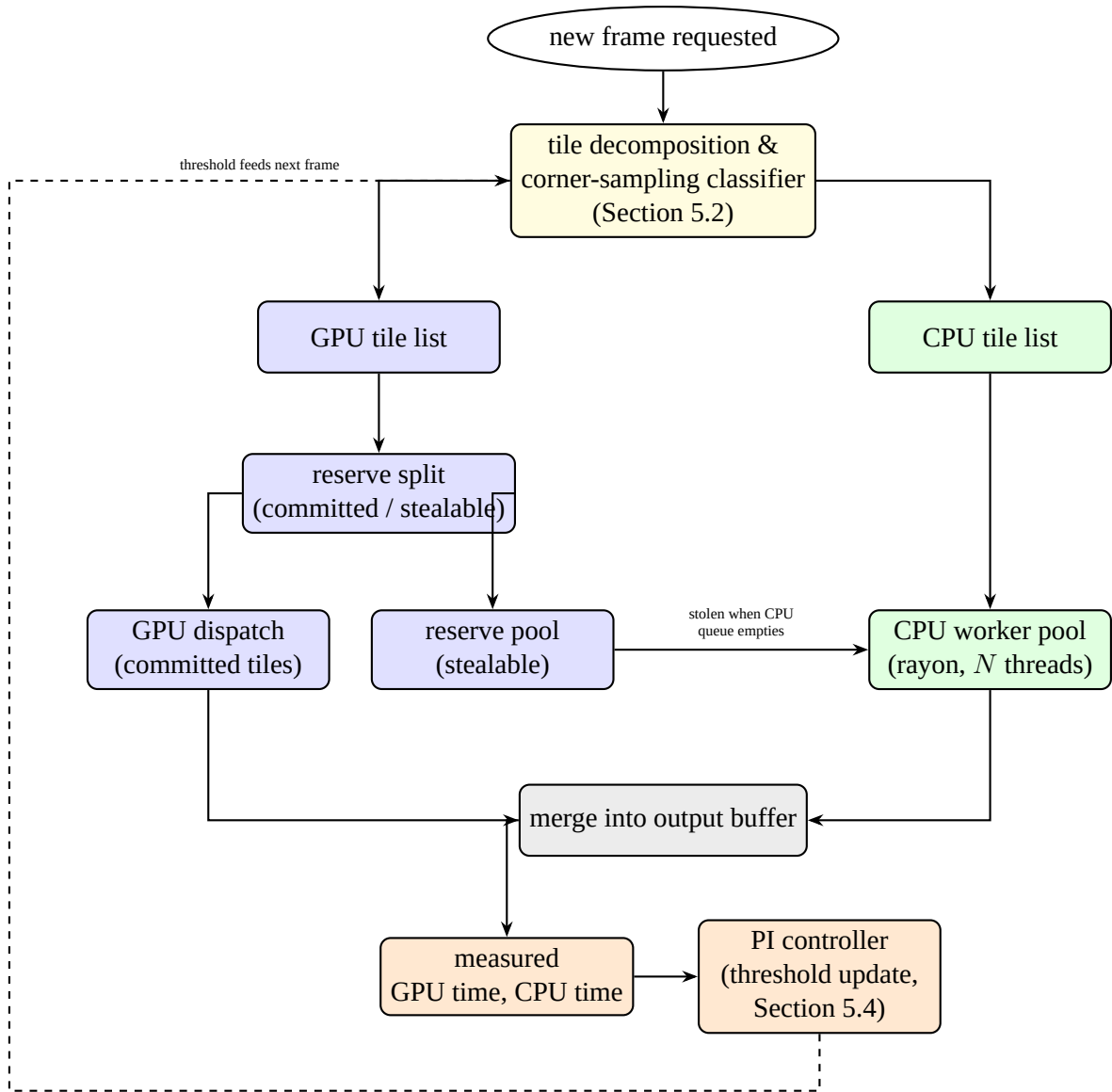


Figure 5.1.1: The hybrid scheduler’s per frame data flow. Yellow is the classifier, blue is the GPU path, green is the CPU path, gray is the merge step, and orange is the cross frame feedback loop that closes through the PI controller. The dashed arrow is the only edge that crosses a frame boundary, every solid edge happens within the frame currently being rendered.

5.2 Tile Decomposition and Classification

The classifier’s job is to decide, for a region of the frame no pixel of which has been rendered yet, whether that region belongs on the GPU or the CPU, and to do so without spending anywhere near as much time as actually rendering it would. It works by corner sampling. The escape time kernel is evaluated at a tile’s four corners only, and how much those four values disagree serves as a proxy for how divergent the pixels inside are likely to be. The frame is first covered by a row major grid of equally sized cells (128×128 pixels by default), classified independently and in parallel, since one cell’s classification touches no shared state and depends on no other cell’s outcome.

Within a cell, the four corner values give a normalized spread,

$$s = \frac{\max(i_1, i_2, i_3, i_4) - \min(i_1, i_2, i_3, i_4)}{N}, \quad (5.2.1)$$

where $i_1, \dots, i_4$ are the four corners' escape time iteration counts and N is the iteration cap, comparable across different iteration caps since it is a fraction of the iteration budget rather than a raw iteration count. If s falls below a threshold, the tile is classified GPU, since its corners agree closely enough that the pixels between them are assumed uniform. That is the same locality assumption Mariani-Silver's border test (Section 3.4) makes about a rectangle's interior, applied here to route work between devices instead of to skip it outright. If s is at or above threshold, the tile is bisected along its longer axis, the same split by longer axis Mariani-Silver's border test uses, into two tiles that are each classified recursively, reusing the two corner samples on the shared edge from the parent and evaluating only the two genuinely new corners the split creates, rather than resampling all four for each half. Recursion bottoms out at a minimum tile size (16×16 pixels by default). A tile that small is classified CPU regardless of its spread, since subdividing further stops paying for itself no matter how divergent the tile still looks. The threshold itself is not fixed. Section 5.4 covers where it comes from.

The classifier's output is two separate lists of tile rectangles, one bound for the GPU and one bound for the CPU, that Section 5.3 dispatches from directly. Measured directly, this whole pass costs 0.03 to 0.32 ms across a shallow to moderate zoom range, negligible next to the frame times it precedes, which run several milliseconds each. Sampling only four corners per tile, rather than every pixel, keeps the cost of deciding where to render small relative to the cost of actually rendering. The split it produces also tracks scene content rather than applying some fixed ratio. On this thesis's benchmark scene, the CPU's share of pixels rises from about 3% at zoom 1 to about 14% at zoom 100, as the set's boundary, the only region divergent enough to fail the corner test, comes to fill more of the frame.

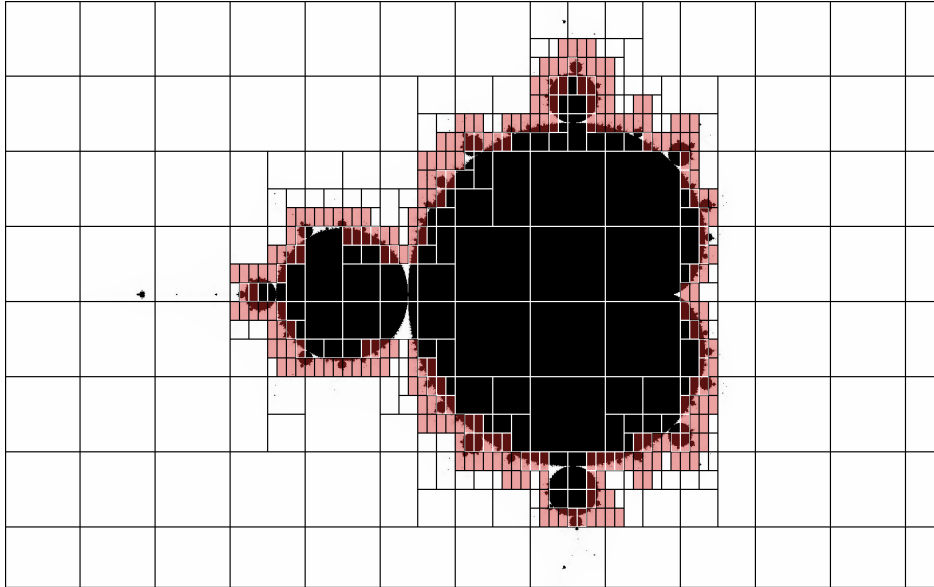


Figure 5.2.1: The classifier's tile partition over a full frame Mandelbrot render. GPU classified tiles are left untinted. CPU classified tiles are shaded red.

Figure 5.2.1 makes that concentration visible. The CPU tiles form a thin band that follows the boundary's own shape rather than a solid mass, and recursion pushes deepest exactly where that shape is least like a straight line. The cusps where the main cardioid

meets its attendant bulbs stand out as small clusters of minimum size tiles within the wider band. That is the same effect behind the 3% to 14% CPU share reported above. However finely the boundary folds at scales below a single pixel, the tile grid can only ever resolve it down to minimum tile resolution, so the CPU's share stays tied to how much of the visible frame the boundary curve passes through, not to how complex that curve actually is.

5.3 Work Distribution & Synchronization

The classifier's GPU list is not committed to the GPU in full. A trailing fraction of it, 20% by default, is split off into a separate reserve pool instead, and the remainder, the committed tiles, is what actually gets dispatched to the GPU immediately. Both the reserve pool and the CPU list, now a work queue of its own, are shared, contended state that any CPU worker thread can draw from.

One worker thread runs per available CPU core, and each worker loops. It claims a tile from its own CPU queue first, and only reaches into the reserve pool once that queue is empty, so GPU reserved work is touched only after a worker's own designated share is exhausted. While those workers run concurrently, the GPU is dispatched on the committed tiles in one batch, so neither device waits on the other for the bulk of the frame. Once that dispatch returns, the scheduler checks how many tiles are still unclaimed in the reserve pool. If at least 64 remain, the CPU workers evidently have not gotten to them yet, still busy with their own queue, so it is worth one more GPU dispatch to finish the remainder rather than wait on the CPU to eventually drain it. Otherwise the remainder is small enough that a second GPU launch's own overhead would not be worth paying, and it is simply left for CPU workers to keep stealing. The GPU readback that follows copies the entire frame buffer regardless of which fraction the GPU actually rendered, since shrinking that copy would require the GPU's share to be one contiguous row range rather than a scattered set of tiles.

Once every CPU worker and the GPU dispatch have both finished, each device's rendered tiles are copied into their true position in a shared output buffer. No per pixel or per tile lock is needed during rendering itself, only the brief moment a worker claims a tile from a queue. The tiles a GPU and a CPU worker render are always disjoint, so the merge step that follows has no contention to resolve.

5.4 Adaptive Load Balancing with a PI Controller

The classifier's threshold (Section 5.2) is a proxy for how divergent a tile's interior is likely to be, not a direct measurement of which device renders that tile faster, and the right proxy value depends on scene content and on the two devices' relative load in ways nothing upstream of rendering can know in advance. A feedback controller instead closes the loop from the outside. It observes each frame's actual measured GPU and CPU wall clock time and nudges the threshold so the two stay balanced on the next frame.

The controller tracks one signed error per frame, $\text{error} = t_{\text{gpu}} - t_{\text{cpu}}$, the difference between that frame's measured GPU and CPU render time, and updates the threshold as

$$\text{threshold} \leftarrow K_P \cdot \text{error} + K_I \cdot \overline{\text{error}}, \quad (5.4.1)$$

with $K_P = 0.0005$ and $K_I = 0.00005$, then clamps the result to $[0.001, 0.5]$. A positive error means the GPU was slower than the CPU that frame, and Equation 5.4.1 responds by lowering the threshold. Since the classifier's own test is $\text{spread} < \text{threshold}$, a lower threshold is a

stricter uniformity requirement, so fewer tiles pass it and more are routed to the CPU on the next frame, relieving the device that was behind. A negative error drives the threshold the other way. The proportional term alone reacts to the current frame only and would leave a steady state bias uncorrected if one device is *consistently* faster rather than just noisily so from frame to frame. The integral term $\overline{\text{error}}$, a moving average over the last 16 frames rather than an unbounded running sum, corrects that residual bias while still letting a single noisy frame's influence age out after sixteen updates instead of persisting forever. The proportional gain is ten times the integral gain, so the controller reacts fastest to the frame it just measured and leans on the integral term only for what that alone does not fix. The $[0.001, 0.5]$ clamp keeps a large, sustained imbalance, such as a GPU driver stall, from driving the threshold to an extreme where the classifier collapses to routing everything to one device regardless of content, which would throw away the awareness of content Section 5.2 was built to provide in the first place.

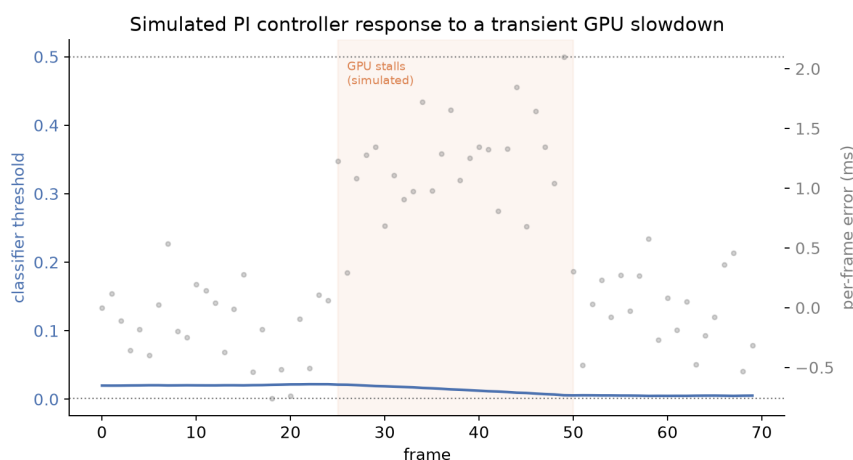


Figure 5.4.1: Equation 5.4.1 simulated over 70 synthetic frames, with a transient GPU stall injected between frames 25 and 50. The threshold falls while the stall lasts, routing more of the frame to the CPU, and the sixteen frame integral window keeps pulling it down for a few frames after the stall ends before the proportional term settles on zero mean noise again. This run happens to reach the 0.001 floor, illustrating exactly the case the clamp above is there to bound.

Because interactive rendering renders the frame again on every pan and zoom, this feedback runs continuously rather than as a one time calibration step. The update runs at the end of every heterogeneous render, and the threshold it leaves behind is exactly what Section 5.2's classifier reads at the start of the very next one. Taken together with the stealing within a frame from Section 5.3, the scheduler corrects for load imbalance on two timescales at once. Stealing absorbs a GPU that runs behind within a single frame, and the controller absorbs a GPU that runs behind *across* frames by reshaping the classification itself. Chapter 7 measures how much of the readback cost this chapter opened with that combination actually recovers, and where the two devices are close enough in speed for the correction to matter at all.

6. EXPERIMENTAL METHODOLOGY

Chapter 7 reports frame time, throughput, and scaling behaviour for every backend built in Chapters 3, 4, and 5. This chapter defines the conditions those numbers were produced under. Section 6.1 covers the software stack and how it is configured and measured. Section 6.2 covers the machine. Section 6.3 covers the fractals, resolutions, and viewports exercised, and Section 6.4 covers the quantities reported.

6.1 Technology Stack

6.1.1 Rust

Rust is a compiled systems language with no garbage collector. Memory safety and freedom from data races are enforced at compile time through ownership and borrow checking, not a runtime collector. That guarantee matters for a renderer whose reported unit is millisecond frame time. A garbage collected implementation would need every measurement split into collection pauses and algorithmic cost before a single number in Chapter 7 meant anything, and this renderer has no such split to make. The compiler lowers to native code through LLVM, so generic and iterator based Rust compiles to the same machine code as an equivalent hand written loop. `unsafe` blocks, used only for CUDA driver calls, are the sole escape from the compiler's checks.

The project targets the 2024 edition and is built through two compile time configurations, applied uniformly to every number in this thesis.

- the release profile, `opt-level = 3`, `lto = "thin"`, thin link time optimization across the whole dependency graph, and `codegen-units = 1`, one codegen unit, trading compile time for the optimizer's full visibility into the program. No measurement in this thesis comes from a debug build, which is an order of magnitude slower and not a usable baseline, and
- `-C target-cpu=native`, set via `rustflags`, letting the compiler target every instruction set the reference machine's CPU (Section 6.2) actually exposes, including the AVX2 instructions the `f32x8/f64x4` SIMD kernels compile down to. A conservative default target would not emit those instructions at all.

Four crates carry the CPU side implementation. This states what each one is used *for*, not how, since Section 3.1 and Section 3.2 already cover the kernels themselves.

- `rayon`, the work stealing thread pool behind every CPU kernel, scalar and SIMD alike,
- `wide`, the portable `f64x4/f32x8` SIMD vector types,
- `num-complex`, the complex number arithmetic the escape time kernels iterate, and
- `image`, writing rendered frames to PNG for output verification.

6.1.2 GPU Technologies

Two GPU technologies back the two GPU kernels compared in this thesis, and they differ in what they let this thesis control at compile time. WebGPU is a portable compute and graphics API, standardized independently of any one vendor. A driver maps it onto whichever native API a platform exposes, Vulkan on the reference machine. Its shaders are written in WGSL and compiled at runtime by the driver, so one kernel source runs unmodified on any WebGPU capable device, at the cost of no compile time control over the machine code the driver’s own compiler ultimately produces. This thesis reaches WebGPU through `wgpu` (version 22), the Rust implementation also used by Firefox and Deno.

CUDA is NVIDIA’s proprietary compute platform. Kernels are compiled ahead of time with `nvcc` into PTX, NVIDIA’s stable intermediate representation, embedded into the compiled binary so nothing needs compiling at runtime. Five flags configure that compilation, applied identically to every CUDA kernel in this thesis.

- `--gpu-architecture=sm_86`, targeting the Ampere generation of the reference GPU (Section 6.2) directly, rather than a generic target the driver would have to translate at load time,
- `-O3`, `nvcc`’s highest optimization level,
- `--ftz=false`, keeping denormalized floats alive, needed for the smooth coloring formula’s accuracy near the escape threshold,
- `--prec-div=true`, full precision division, and
- `--fmad=false`, disabling fused multiply add contraction so CUDA’s arithmetic rounds exactly like the CPU kernels’ does, at a measured cost under 1% of kernel throughput once other precision costs are removed (Section 4.2).

This thesis reaches CUDA 12 through `cudarc`, a Rust binding to the CUDA driver API.

6.1.3 Benchmarking Technology

Three tools measure the renderer, each suited to a different purpose.

- `criterion`, a statistical benchmarking crate, runs the primary suite (`fractal_bench`). Each group gets a two second warm up, discarded, then five seconds of measurement, and reports a mean with a 95% confidence interval rather than one sample. Repeated runs of a nominally identical kernel have disagreed by more than 50% within a single `criterion` invocation.
- `bench_runner`, a purpose built binary that renders one fixed configuration deterministically, for pairing with `perf stat` or for emitting machine readable JSON, and
- a set of standalone probe binaries (`gpu_probe`, `ptx_variants`, `sched_probe`, `sched_overhead`, `sched_deepzoom`, `tilled_dispatch_probe`, `adapters`), each isolating one phase or one hypothesis that a single frame time number cannot separate on its own, for example kernel time from host readback time, one compiler flag’s effect from another’s, or scheduler overhead from scheduler decision quality.

`perf stat` supplies CPU cache and branch counters, NVIDIA Nsight Compute supplies GPU occupancy and warp efficiency counters, and RAPL package energy counters together with `nvidia-smi`’s power draw supply the energy figures in Section 6.4.4.

The four existing renderers surveyed in Section 2.2 share none of this tooling. Each was instrumented with an equivalent two layer harness for this thesis, a Google Benchmark based `criterion_bench` target and a `bench_runner` CLI call that project’s own compiled kernel directly, so all five renderers run from one command line and produce comparable JSON. Each renderer’s own threading model is measured as built, not normalized to rayon’s pool. The comparison targets algorithmic throughput per pixel actually computed, so any shortcut a project uses to avoid computing a pixel outright, such as XaoS’s boundary tracing or FractalNow’s quad interpolation, is disabled. Both substitute interpolation for computation on part of the image, which would measure how well a project guesses rather than how fast its kernel runs.

6.2 Hardware

All measurements in this thesis, referred to throughout as the reference machine, come from one laptop, built around the following.

- an AMD Ryzen 7 5800H CPU, eight Zen 3 cores, sixteen threads via two way SMT,
- an NVIDIA GeForce RTX 3050 Laptop GPU, Ampere (sm_86), CUDA 12.0, driver 580.173.02, PCIe 3.0 x8 (8 GB/s theoretical), and
- an integrated AMD Radeon Vega GPU, code name Renoir.

Figure 6.2.1 lays out how these components actually connect. The Ryzen die carries the Radeon Vega integrated GPU on package, so both share the same dual channel DDR4 system RAM through one on die memory controller, with no PCIe transfer between them at all. The RTX 3050 is a separate, discrete part reached only over PCIe 3.0 x8, with its own dedicated GDDR6 VRAM the CPU cannot address directly, the link Section 4.3 and Chapter 5 spend so much of this thesis working around the cost of.

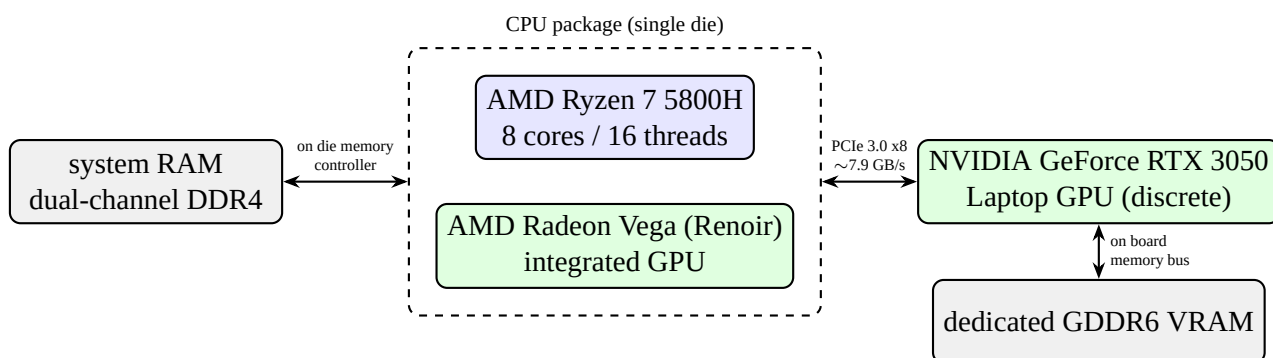


Figure 6.2.1: The reference machine’s component topology. The integrated GPU shares the CPU package and system RAM at no transfer cost. The discrete GPU is reachable only over PCIe, with its own separate VRAM.

`wgpu`’s adapter selection falls back silently to the integrated GPU when no NVIDIA Vulkan driver is installed, understating GPU readback cost, since the integrated part shares memory with the CPU and pays no PCIe transfer at all. Every benchmark session logs the selected adapter at startup, checked before its numbers are used.

The reference machine is a laptop, not a dedicated bench machine, and is thermally variable. Identical CPU or GPU benchmarks, run in separate sessions with no code change

between them, have moved 15 to 30% from thermal state alone. Every ratio and speedup figure in Chapter 7 is computed within a single run. Absolute millisecond figures carry that variance regardless.

6.3 Benchmarks and Test Scenes

6.3.1 Fractals, Resolutions, and Iteration Counts

The Mandelbrot set is this thesis’s sole benchmark target. The renderer also implements Julia, Newton, and Nova fractals on the same escape time machinery, but every optimization in Chapters 3 through 5, the cardioid and bulb tests, the distance estimate, the Mariani-Silver subdivision, and the SIMD and GPU kernel work, was developed and measured against Mandelbrot specifically, and that is what the rest of this thesis reports.

The benchmark suite exercises Mandelbrot at three resolutions, 800×600 , 1920×1080 , and 3840×2160 , chosen to confirm that a technique’s benefit holds independently of frame size. Unless a sweep states otherwise, `max_iter` is fixed at 1000. The one exception is a convergence check against the Mandelbrot set’s area, estimated in the literature at approximately 1.50659177, which sweeps `max_iter` itself, from 100 to 20000.

6.3.2 Viewports

Most benchmarks render the whole visible set from its natural center ($c = -0.5$, zoom 1), mixing interior, boundary, and fast escaping pixels in roughly the proportions an interactive session shows. Where a technique’s cost depends specifically on how much of the frame is boundary rather than interior, the suite instead uses five fixed sample points spanning the main cardioid, the largest bulb, a minibrot, and a filament dense region, so a result cannot be an artifact of one convenient view with little boundary in it.

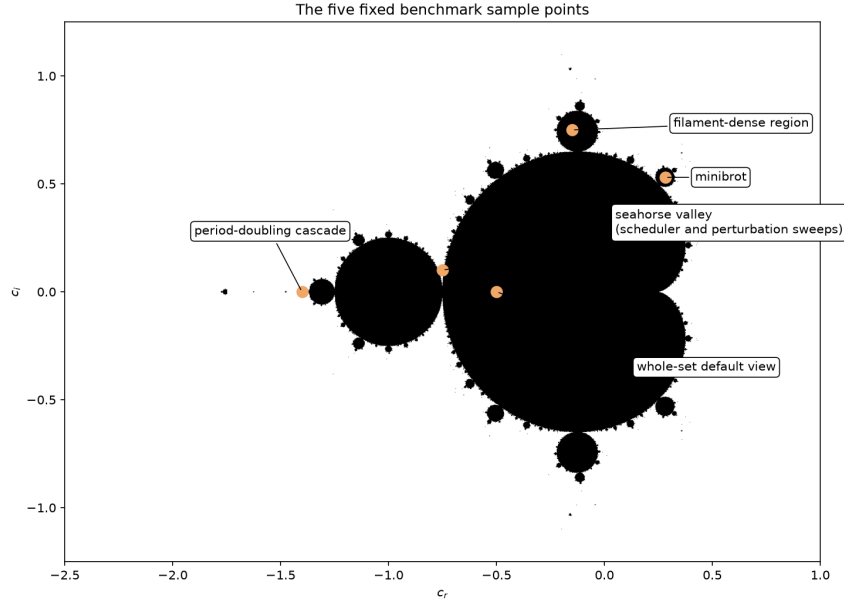


Figure 6.3.1: The five fixed sample points, chosen to span a whole set view, a filament dense boundary region, a minibrot, and the period doubling cascade toward the Feigenbaum point along the negative real axis. The seahorse valley point doubles as the reference center for the zoom sweeps in Section 6.3.3.

6.3.3 Zoom Sweeps

Three sweeps probe zoom dependent behaviour. A shallow sweep (zoom 1, 10^2 , 10^4) stays below the 10^6 threshold where both CPU and GPU kernels hand off from f32 to f64. A deep sweep (zoom 10^4 to 10^{15}) crosses that threshold, at a fixed seahorse valley reference point ($c = -0.75 + 0.1i$) chosen because its boundary stays dense enough, at every zoom level in the sweep, to give the tile classifier real work. A third sweep (zoom 1 to 10^{12} , same reference point) evaluates perturbation theory against plain f64.

6.3.4 Cross Renderer Comparison

The comparison against the four existing renderers is restricted to what all five projects share, the Mandelbrot kernel, at matched resolution, iteration count, and view, measured as raw per pixel iteration time rather than full colored frames, since the renderers differ in how much color mapping work is fused into that step.

6.4 Metrics

6.4.1 Frame Time and Throughput

Frame time is reported as `criterion`'s sample mean with its confidence interval. Throughput, pixels rendered per millisecond, comes from the same measurement via `criterion`'s element count facility rather than being derived afterward. A flat throughput curve across resolutions is this thesis's working definition of an embarrassingly parallel kernel. A curve that bends indicates a cost scaling with frame size rather than pixel count alone.

6.4.2 Compute Efficiency

Cross kernel comparisons are additionally stated in GFLOPS, under one fixed convention. Each iteration of the canonical $z = z^2 + c$ update counts as eight floating point operations, with no credit for fused multiply add contraction. The convention is a simplification, not a literal instruction count, but applying it uniformly, to this renderer’s own kernels and to all four competitor projects, is what makes a GFLOPS figure comparable across five independently written codebases.

6.4.3 Precision Controlled Comparison

Every reported speedup states the precision, f32 or f64, both sides ran at. Pairing a fast f32 GPU kernel against a slower f64 CPU baseline inflates the apparent gain independently of either device’s real throughput, the effect Lee et al. document for CPU versus GPU comparisons generally [10]. Every GPU versus CPU ratio in this thesis is reported at least twice, once against a naive scalar baseline, and once against the CPU’s own best available kernel at matching precision.

6.4.4 Parallel Scaling and Energy

Thread scaling is reported as speedup and parallel efficiency relative to that same renderer’s own single threaded run, never across renderers, since threading models differ structurally between projects. Energy efficiency is reported separately from speed, as gigaflops per watt, from RAPL package energy counters (CPU) and `nvidia-smi` power draw (GPU) over a sustained run, since a slower device is not necessarily a less efficient one.

6.4.5 Correctness and Microarchitectural Metrics

Two further metrics apply where the question is not about speed. Where an optimization is expected to change performance only, never output, correctness is checked as a per pixel difference count between kernel variants. Zero differing pixels across the full frame is the bar any such change must clear before its speed number is reported. Where a hypothesis concerns GPU execution behaviour specifically, for instance whether boundary heavy regions cause more warp divergence than uniform ones, warp occupancy and warp efficiency, read directly from Nsight Compute, settle it independently of whether the effect is large enough to move frame time.

7. RESULTS AND DISCUSSION

This chapter presents the measurements the setup in Chapter 6 produces. Every figure comes from the reference machine defined there, and every ratio quoted here follows the precision controlled and within run comparison rules Section 6.4 sets out. Most figures in this chapter are generated directly by this project’s own `aggregate_bench.py` tooling, described in Section 6.1, from a single archived benchmark run rather than redrawn by hand, so a number that looks slightly different from one already quoted in an earlier chapter is run to run thermal variance, the kind Section 6.2 describes, not a contradiction. The chapter follows the order Chapters 3 through 5 introduced the underlying techniques in, the CPU baseline and SIMD first, then the two GPU backends, the end to end pipeline, the hybrid scheduler, perturbation theory, and finally the cross project and numerical validation results. Two results carry more weight than the rest. Figure 7.2.1 in Section 7.2 is the single measurement every earlier optimization chapter feeds into, render throughput for every backend this thesis built, side by side, and Section 7.4 is where the hybrid scheduler, this thesis’s central contribution, is checked against the alternative of just picking the faster device and against the split it should theoretically be able to match.

7.1 CPU Baseline and SIMD Scaling

Figure 7.1.1 confirms the thread scaling Section 3.1 describes. `ruster`’s rayon parallel row chunking stays close to a straight line on the log scale plot through eight threads and only bends at sixteen, the signature of simultaneous multithreading, where a second logical thread on an already busy physical core buys a fraction of a full core rather than another doubling. `FractalRendererCpp` scales worse at every thread count, partly because it spawns and joins a fresh `std::thread` per frame rather than reusing a pool, an implementation cost noted in Section 6.3.4 that this comparison folds into the per-thread number rather than isolating it.

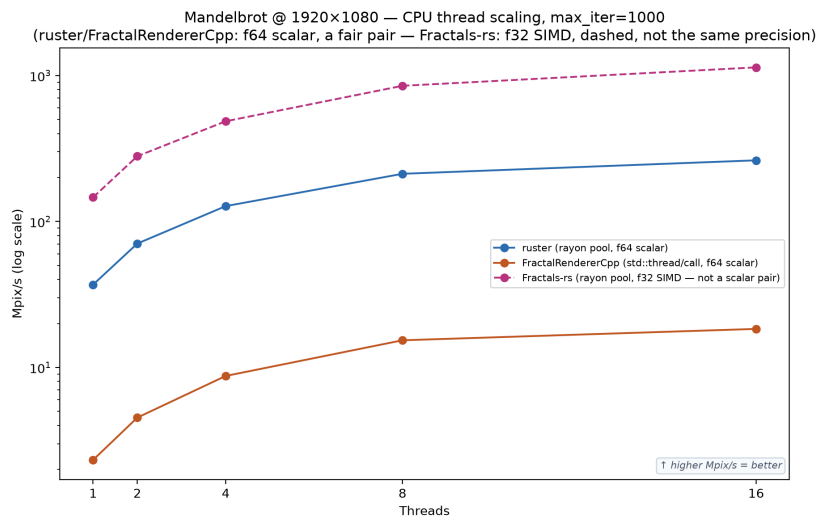


Figure 7.1.1: CPU thread scaling, `ruster` vs `FractalRendererCpp`, Mandelbrot 1080p, f64 scalar on both sides.

Figure 7.1.2 shows the SIMD kernel progression from Section 3.2, measured at 1080p and 4K. Each step, $f64 \times 4$, $f32 \times 8$, then the two chain $f32 \times 8 + \text{ILP}$ kernel, moves the frame time down, and the pattern repeats almost identically at both resolutions, evidence that the gain is a property of the kernel rather than of one particular frame size. The speedup from scalar to $f32 \times 8 + \text{ILP}$ is $1.92 \times$ at 1080p and $2.37 \times$ at 4K, with the instruction level parallelism step contributing under a tenth of that. Section 3.2 attributes the small residual to the cardioid and bulb tests already pruning most of the pixels that would otherwise fill the second chain's pipeline.

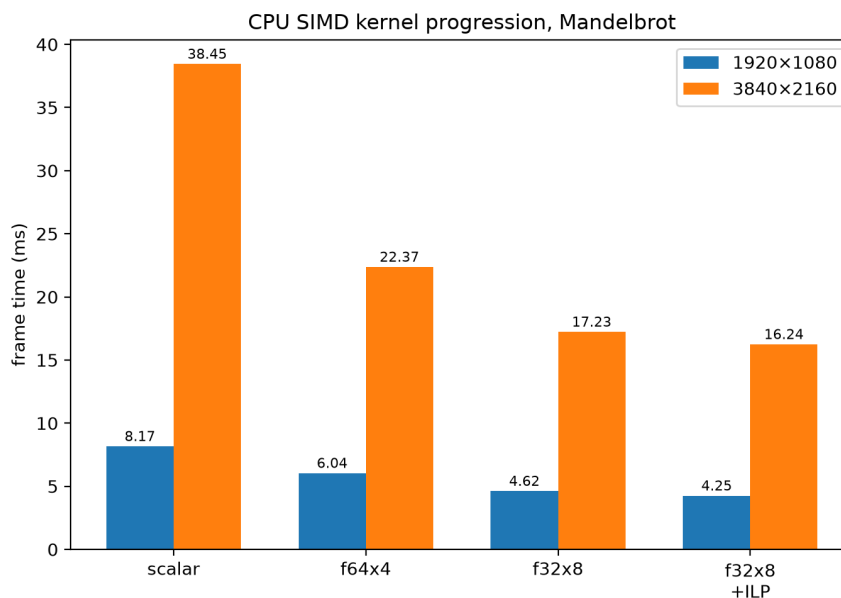


Figure 7.1.2: CPU SIMD kernel progression, Mandelbrot, 1080p and 4K.

Figure 7.1.3 checks two more CPU side choices against the plain rayon row chunked baseline. The left panel is the traversal order question from Section 3.9, Hilbert tiled traversal against plain row major, and the measured gap matches that section's claim directly. Hilbert renders slower at every resolution tested, roughly 224 against 133 Mpix/s at 800×600 and the gap narrowing but not closing by 3840×2160 , with no compensating cache locality win to show for the extra per pixel bit interleaving cost. The right panel is the instruction level parallelism question from Section 3.2 on its own, the interleaved $f32 \times 8 + \text{ILP}$ kernel against plain $f32 \times 8$, and the interleaved kernel leads at every resolution tested, from a narrow 3% margin at 800×600 to roughly 9% at 1080p, consistent with a benefit that needs enough independent per pixel work in flight to be worth the second chain's bookkeeping.

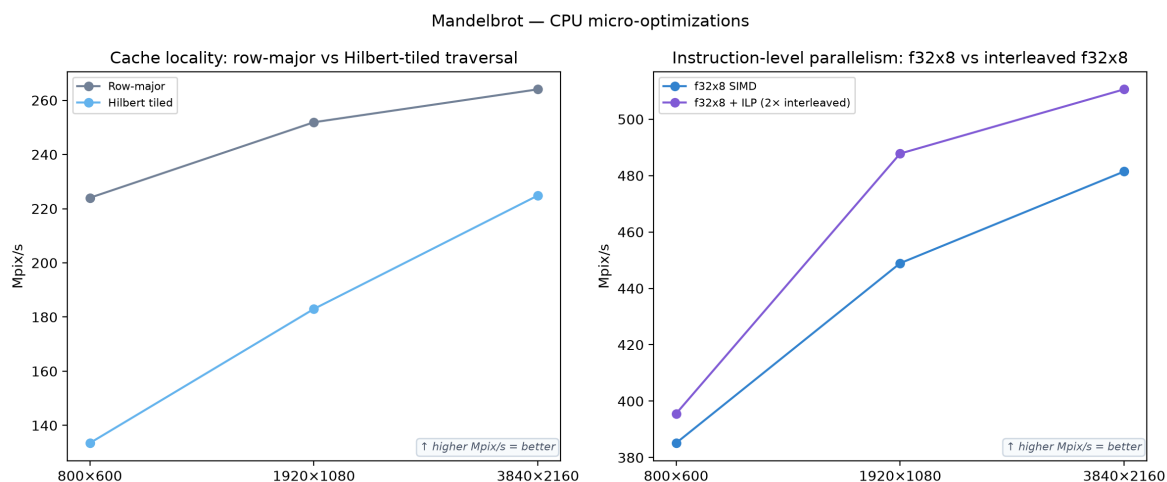


Figure 7.1.3: CPU micro optimizations, Hilbert vs row major traversal and plain vs interleaved f32x8, Mandelbrot.

7.2 GPU Backends

This section’s first figure is the chapter’s centerpiece. Every optimization measured exists to move one number, how many pixels a backend renders per second. Figure 7.2.1 puts every backend this thesis built on the same axes, CPU scalar, CPU SIMD, CUDA, wgpu, and all four competitor projects from Section 6.3.4, Fractals-rs, FractalRendererCpp, XaoS, and FractalNow, at three resolutions, so the whole CPU to GPU progression is visible in one place rather than scattered across a chapter’s worth of separate panels. XaoS and FractalNow are measured on their raw per pixel kernel with boundary tracing and quad interpolation forced off, as Section 6.3.4 explains, not on a measurement of either project’s real interactive engine. A second, hatched XaoS bar turns boundary tracing back on: 50.8, 85.3, and 114.4 Mpixels/s across the three resolutions, 21× to 36× its own raw kernel and already ahead of its own multithreaded raw-kernel bar despite running on a single thread, since `btrace.cpp`’s threaded path is disabled upstream. It is a single static frame with no previous frame to guess from, so it measures the boundary-trace-and-flood-fill algorithm alone, not the additional frame-to-frame reuse XaoS’s real zoom animator adds on top, and is not comparable to any other, multithreaded bar in this figure.

CUDA leads every other backend at every resolution, wgpu comes second, and the gap between the two GPU backends stays roughly proportional to frame size, consistent with a fixed per pixel kernel cost rather than a fixed per-frame overhead dominating either one. Peak throughput, at 3840×2160 , reaches roughly 1389 Mpixels/s on CUDA. The two red hybrid scheduler bars exist only at 1080p, since that benchmark group sweeps zoom at one fixed resolution rather than resolution at one fixed zoom, and Section 7.4 reads them directly rather than through this figure.

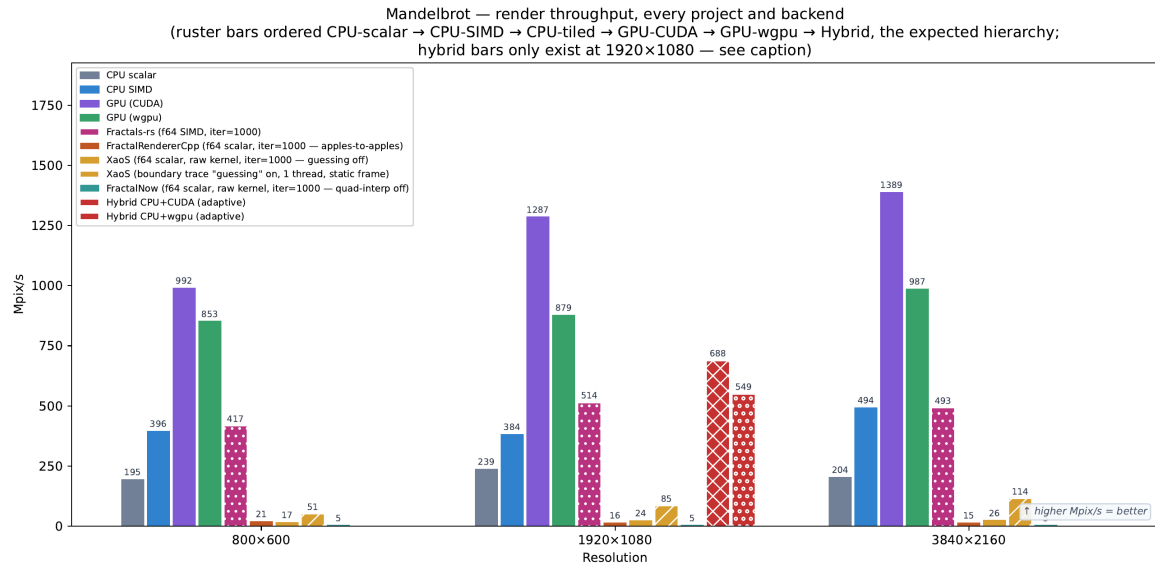


Figure 7.2.1: Render throughput, every ruster backend and all four competitor projects, Fractals-rs, FractalRendererCpp, XaoS, FractalNow, three resolutions, Mandelbrot, zoom 1.

The headline measurement of this chapter. Every CPU and GPU optimization from Chapter 3 through Chapter 4 shows up here as the distance between one bar and the next. The hatched XaoS bar is its real boundary-trace "guessing" algorithm, single thread, single static frame, not comparable to the multithreaded bars around it.

Read left to right within any one resolution group, the bars trace the whole optimization path this thesis takes. Scalar to SIMD is Chapter 3's work, SIMD to either GPU backend is Chapter 4's. CUDA's lead over CPU SIMD, roughly $2.5\times$ to $3.4\times$ depending on resolution, is the return on building a GPU backend at all, and the rest of this section explains where inside that CUDA number the time actually goes.

Figure 7.2.2 isolates why CUDA wins the frame. The two kernels are within 5% of each other, but host readback is 83% of the CUDA frame and 88% of the wgpu frame, and wgpu pays roughly 0.8 ms more of it, consistent with the extra device to device staging copy that CUDA's single `dt oh_sync_copy` avoids, as Section 4.2 explains.

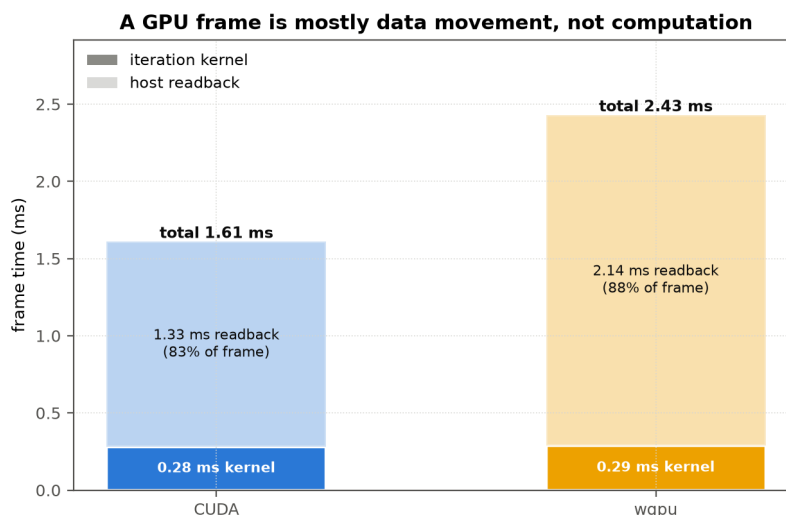


Figure 7.2.2: GPU frame time breakdown, kernel vs host readback, CUDA vs wgpu, 1920×1080 , zoom 1.

Both GPU backends, in other words, spend most of a frame moving pixels back to the host, not computing them. That reframes Section 7.3’s question before it is even asked. If a GPU backend is already dominated by data movement rather than iteration, adding a CPU side `colorize()` stage on top can only make that imbalance more visible, not less, unless that stage moves to the GPU too, which, for CUDA, it has, as Section 4.3 describes.

7.3 End to End Pipeline

Figure 7.3.1 adds the `colorize()` step as measured before either GPU backend could skip it, run on the CPU for every backend alike. The GPU’s advantage over the CPU narrows sharply once that fixed 2.72 ms stage is included, from roughly $5.4\times$ on the render stage alone to $2.09\times$ for CUDA and $1.72\times$ for wgpu end to end. Of a 5.19 ms CUDA pipeline, roughly 0.4 ms is fractal iteration, and the remaining 4.8 ms is data movement and coloring. That is exactly what made moving `colorize()` onto the GPU the single highest leverage optimization this measurement pointed at, larger than the fp64 bulb fix in Section 4.2 and larger than anything the scheduler in Chapter 5 can recover. CUDA’s `colorize()` has since moved on device through `colorize_kernel` and `render_and_colorize`, described in Section 4.3, pixel exact against the CPU version and 54% to 63% faster end to end at the shallow and mid zooms where `colorize` used to dominate frame time. wgpu and the heterogeneous scheduler still `colorize` on the CPU. The scheduler does so by necessity, since its merged buffer only ever exists on the host side, as Chapter 5 explains, so the gain above remains open for those two paths.

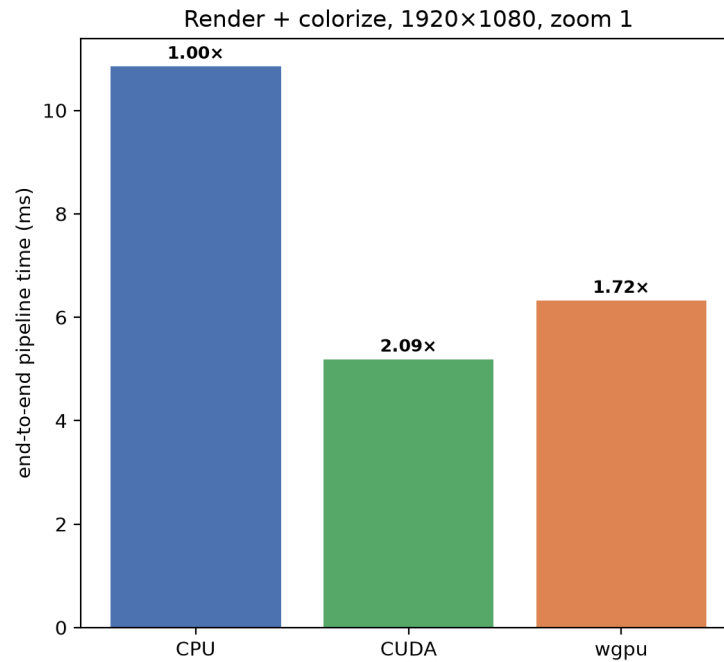


Figure 7.3.1: End to end pipeline time, render plus colorize, 1920×1080 , zoom 1. Measured with colorize run on the CPU for every backend, before CUDA’s own colorize step moved on device, see prose.

Figure 7.3.2 breaks the same stage down per backend, render only against render plus colorize, from that same measurement where every backend colorized on the CPU. The fixed `colorize()` cost took a bigger proportional bite out of a GPU backend than out of the CPU, since it was the same CPU side stage every time, roughly a fifth off the CPU’s own throughput but closer to two thirds off either GPU backend’s, the same Amdahl effect Figure 7.3.1 already shows stated as an absolute time rather than a proportion. This is precisely the effect that made moving colorize onto the GPU worth doing for CUDA.

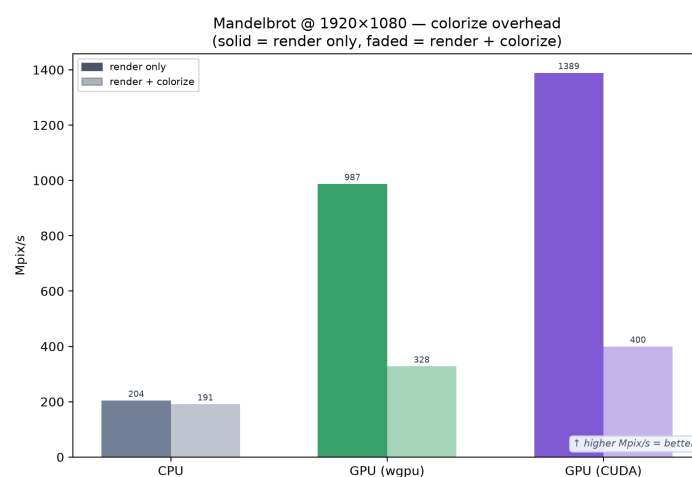


Figure 7.3.2: Colorize overhead per backend, render only vs render plus colorize, Mandelbrot.

Chapter 5’s scheduler exists to buy back exactly the time this section just spent two figures accounting for, so that is where this chapter turns to next.

7.4 Hybrid CPU+GPU Scheduler

This section is where the thesis’s central claim gets tested against data. Chapter 5 built a scheduler that watches both devices at runtime and routes tiles by measured cost instead of dividing a frame blindly. The two figures here are the evidence for whether that machinery earns its keep, at a shallow zoom where it should struggle and at a deep zoom sweep where it is designed to win.

Figure 7.4.1 measures the scheduler from Chapter 5 against both solo backends and the naive static split, at zoom 1. The static 50/50 CPU plus wgpu split, with no classification at all, is the worst active configuration on the chart, 408 against wgpu’s own 879 Mpix/s solo. That is exactly the failure Section 5.3 motivates the corner sampling classifier by, an even split leaves the faster device waiting on the slower one. The adaptive scheduler recovers most of that loss, 688 Mpix/s on CUDA and 549 on wgpu, but still trails using either GPU alone. Every scheduler configuration loses to simply using the better GPU alone at this zoom, the Section 5.1 arithmetic showing through directly, its own machinery costs a fixed amount, and the kernel time it can shrink is a small fraction of an already GPU dominated frame. At shallow zoom, in short, there is not enough imbalance between devices for a scheduler to fix.

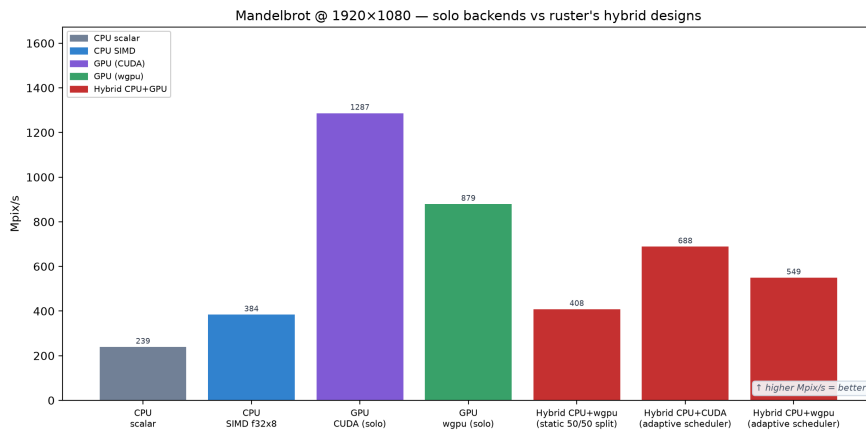


Figure 7.4.1: Solo backends vs the static split and the adaptive scheduler, Mandelbrot 1080p, zoom 1.

Deep zoom is a different regime, and it is where this thesis’s central result actually lives. Figure 7.4.2 extends the same comparison from zoom 10^4 to 10^{15} . Right at the 10^6 precision threshold Section 6.3.3 sweeps around, CUDA’s own fallback from f32 to f64 lands it briefly above the CPU, 218 against 130 ms, before both devices settle within about $1.1\times$ of each other from 10^7 onward. That crossing is exactly where the scheduler pays off. At zoom 10^6 it beats not only the better solo backend, the CPU at that specific zoom, but the ideal proportional split too, 74.3 against 81.6 ms, a $1.75\times$ win over the better device that is direct evidence for this thesis’s central claim, corner sampling classification routes expensive tiles to the CPU instead of dividing the frame blindly, and that routing is worth more than an even split even when an even split is the theoretically available ceiling. The wins past 10^7 are smaller, roughly $1.13\times$ to $1.15\times$, and leave roughly 43% of the available headroom on the table by the load balancer’s own numbers, since the ideal split reaches roughly 45 ms at those zooms while the hybrid scheduler still measures near 79 ms, a tuning problem in a regime this benchmark suite only now covers, not a structural one.

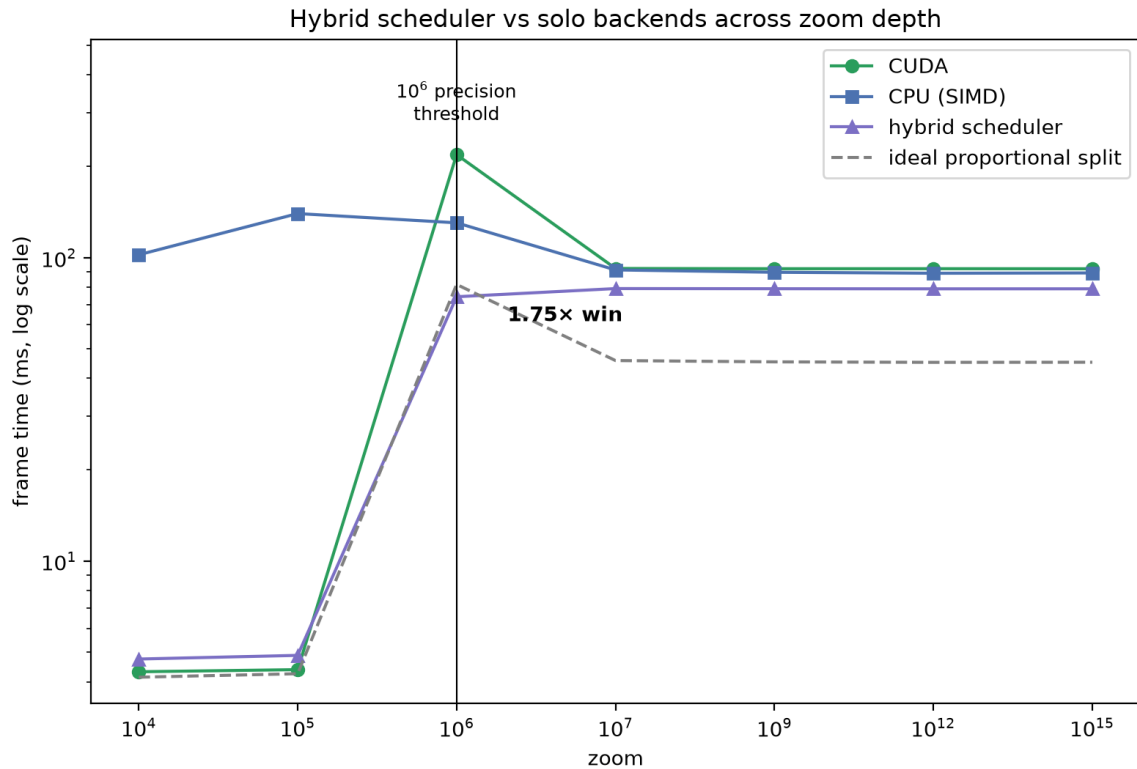


Figure 7.4.2: Hybrid scheduler vs solo backends and the ideal proportional split, across zoom depth, Mandelbrot, seahorse valley. The $1.75\times$ win at 10^6 is this thesis’s clearest evidence that corner sampling classification, not just having two devices, is what the hybrid scheduler contributes.

The two figures together answer the question Chapter 5 opened with. A scheduler is only worth its complexity where devices are imbalanced enough for routing to matter. It loses at shallow zoom because there is nothing to route around, and it wins precisely where Section 5.1 predicted it would, at the precision cliff where one device suddenly costs far more per tile than the other.

7.5 Perturbation Theory

Figure 7.5.1 measures perturbation theory, described in Section 3.6, against the plain direct kernel, per backend, from zoom 1 to 10^{12} . Perturbation never wins in this range, on any of the three backends. The gap is starkest on CUDA, where the naive kernel’s own f32 fast path renders the shallow end of the sweep at well over 1000 Mpix/s while perturbation, restricted to f64 throughout as Section 4.2 explains, stays flat near the bottom of the chart the whole way across. That confirms Section 3.6’s own conclusion directly, this sweep never reaches a zoom where plain f64 actually breaks down, so perturbation is pure overhead here rather than the correctness feature it becomes only at zoom levels beyond what any backend’s direct kernel can represent at all.

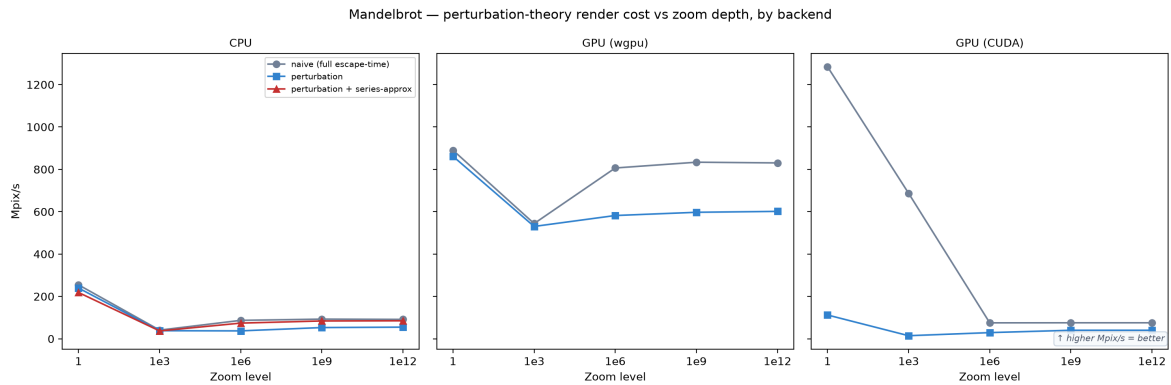


Figure 7.5.1: Perturbation theory vs the plain direct kernel, per backend, across zoom depth, Mandelbrot.

Figure 7.5.2 explains why that overhead is cheap enough to be a reasonable trade once it is needed. The double-double reference orbit from Section 3.8 costs roughly three times its plain f64 counterpart per call, 0.0010 against 0.0003 ms, and the series approximation coefficients from Section 3.7 grow roughly tenfold from shallow to deep zoom. Both numbers stay well under a microsecond, negligible next to a millisecond scale frame, which is exactly why Section 3.8 can afford to pay the double-double cost once per frame rather than once per pixel.

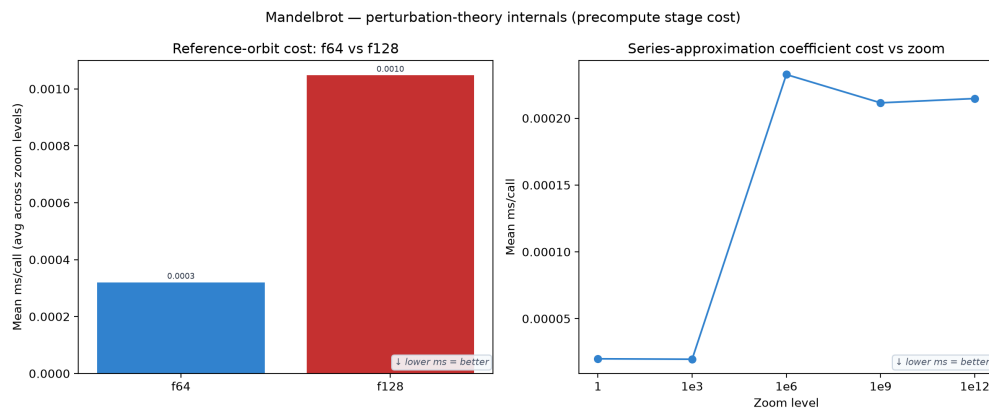


Figure 7.5.2: Perturbation precompute cost, f64 vs f128 reference orbit and series approximation coefficients vs zoom.

Perturbation theory costs almost nothing to keep switched on, then, even in the benchmark range where it never pays for itself. The next section turns from rustur's own backends to how the whole renderer stacks up against the four competing projects Section 2.2 surveyed.

7.6 Cross-Project Comparison

Figure 7.6.1 places rustur against the three competitor renderers across every metric class this thesis measures for them, single pixel kernel cost, full frame render at three resolutions, thread scaling from one to sixteen threads, and, where available, the full pipeline, all under the protocol Section 6.3.4 sets out, matched resolution, iteration count, and view. rustur leads every panel, by roughly one order of magnitude over the fastest competitor and two over the slowest, at every thread count and every resolution tested. The pipeline panel only has data for FractalRendererCpp and FractalNow, since XiaoS's and FractalNow's own

”guessing” shortcuts, described in Section 6.3.4, apply to a full colored frame, not to ruster’s split render and colorize measurement, so the two are not directly comparable there.

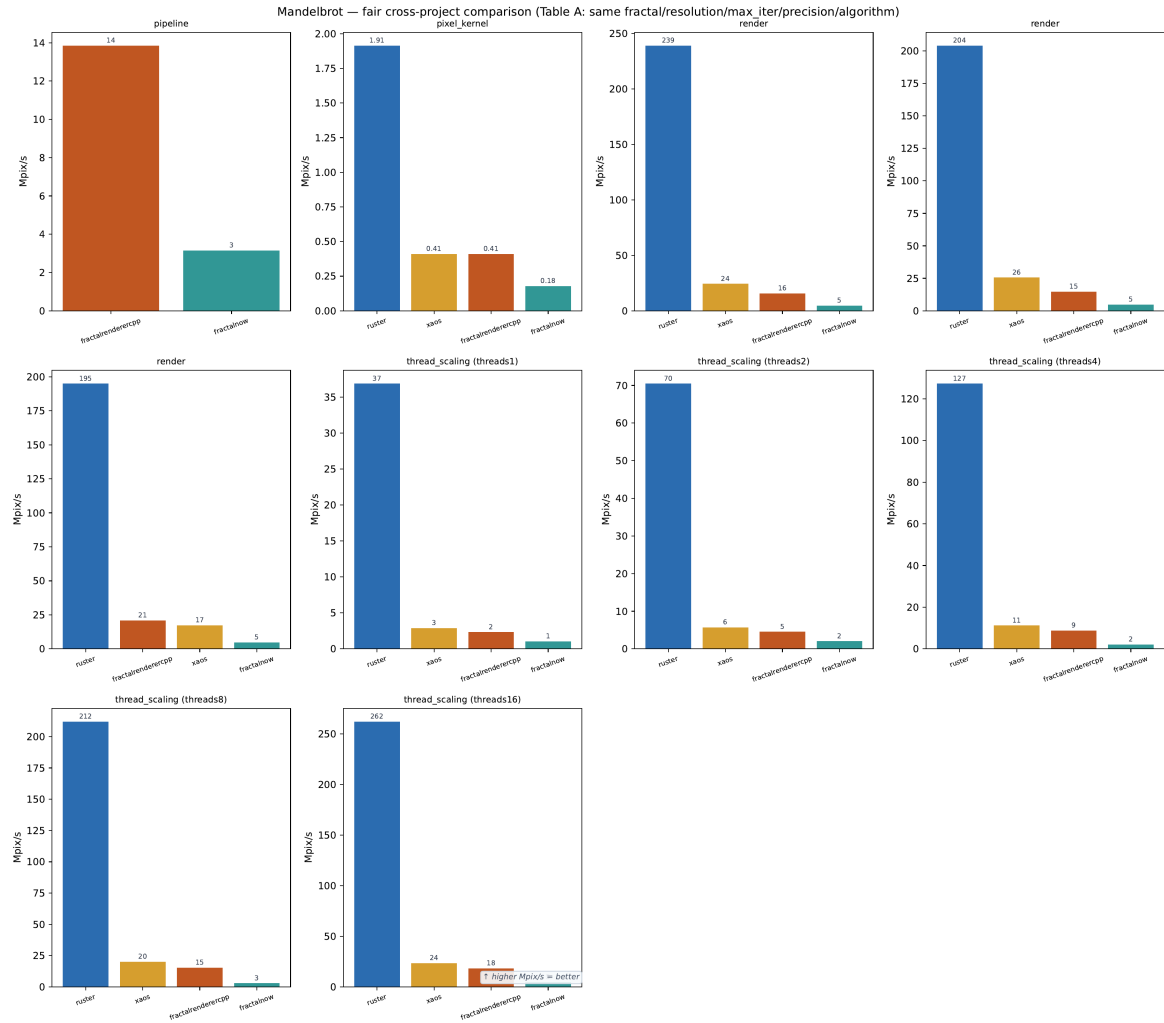


Figure 7.6.1: Fair cross-project comparison, matched fractal, resolution, max_iter, precision, and algorithm class.

Figure 7.6.2 isolates just the per pixel kernel, normalized to evaluations per second so that ruster’s multi sample point calls and the other projects’ single point calls remain comparable, as Section 6.1 explains. ruster’s kernel evaluates roughly $4.7\times$ faster than Xaos’s raw kernel, its own real time boundary tracing forced off, and by coincidence lands at almost exactly the same margin, roughly $4.7\times$, over FractalRendererCpp’s, whose bailout test resquares z instead of reusing the update step’s own intermediates. FractalNow’s raw kernel, quad interpolation forced off, is slowest of the four, a generic macro generated dispatch layer, as Section 6.3.4 explains, rather than any specific algorithmic shortfall.

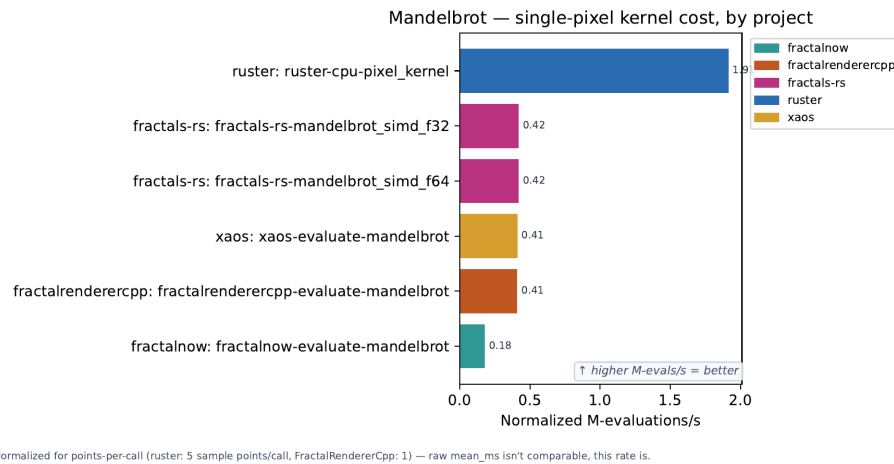


Figure 7.6.2: Single pixel kernel cost, four projects, Mandelbrot, normalized M-evaluations/s.

The single threaded comparison against FractalRendererCpp, the fairest pairing available since it controls for thread count, precision, and algorithm class simultaneously, reaches $15.9\times$, 56.2 against 893.0 ms, not from a language advantage but from ruster's kernel already carrying the cardioid, period 2, and period 3 rejection tests and Brent cycle detection that FractalRendererCpp's naive loop has none of. Figure 7.6.3 shows the corresponding resolution sweep for FractalRendererCpp on its own, throughput flat within noise across all three resolutions, the same embarrassingly parallel signature Section 6.4.1 defines. Fractals-rs, run in the same session as the other four, is SIMD only and shows the same flat signature on both of its precision paths. Its fast f32 path runs 811 to 1113 Mpix/s across the sweep and its high f64 path 417 to 514 Mpix/s, each within noise of itself at every resolution rather than trending with frame size.

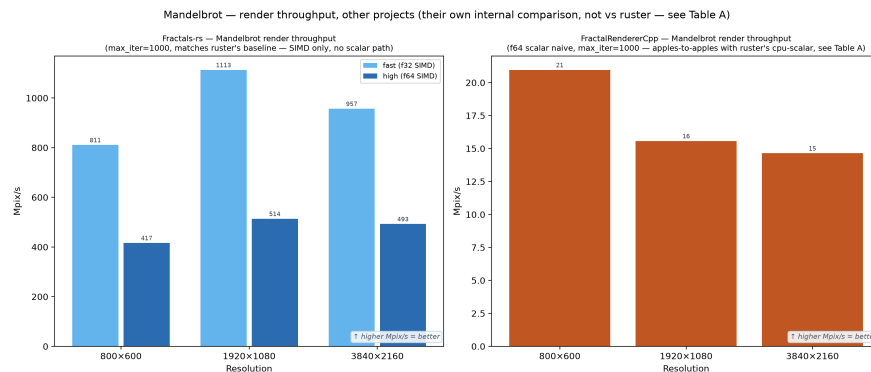


Figure 7.6.3: Fractals-rs's own resolution sweep, f32 vs f64 SIMD, and FractalRendererCpp's own resolution sweep.

Every comparison in this section has been about speed. The next one asks a question speed alone cannot answer, which of ruster's own two backends is cheaper to run.

7.7 Energy Efficiency

Figure 7.7.1 answers a question frame time alone cannot, which device is cheaper to run, not just faster. The GPU reaches 54.37 GFLOPS at 13.49 W against the CPU's 100.35 GFLOPS at 33.82 W, so the CPU wins on raw throughput but the GPU is roughly 36% more

efficient per watt, 4.03 against 2.97 GFLOPS/W. Faster and more efficient are answers to different questions on this hardware, and a deployment optimizing for battery life or thermal budget rather than latency would read this figure differently than the rest of this chapter does.

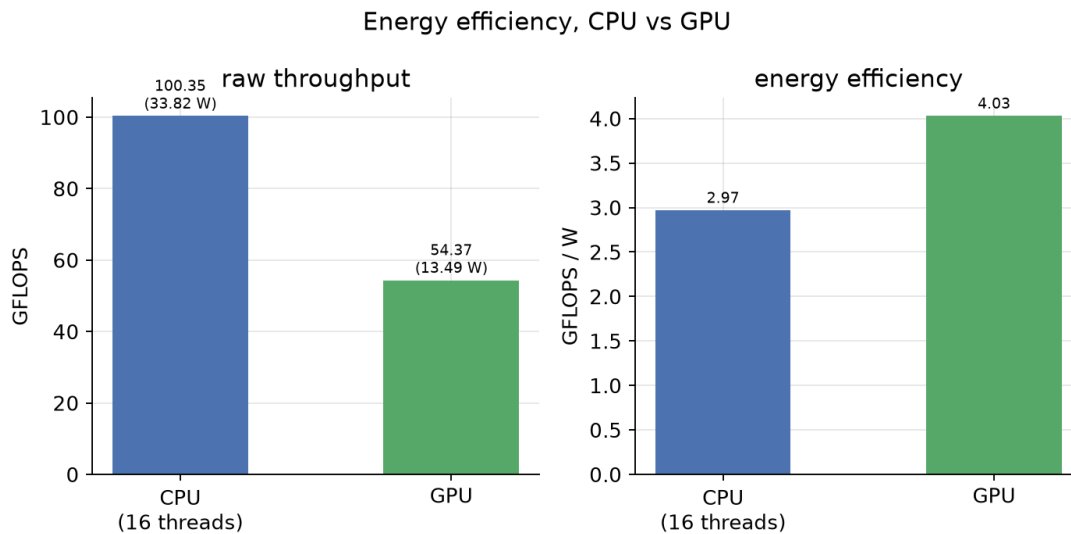


Figure 7.7.1: Energy efficiency, CPU vs GPU, raw GFLOPS and GFLOPS per watt.

7.8 Numerical Validation

Every earlier section in this chapter asked how fast the renderer is. This one asks whether it is still right. Chapters 3 through 5 rewrote the same hot loop a dozen times over, scalar to SIMD, CPU to GPU, direct iteration to perturbation, and a fast renderer that quietly computes the wrong set would be worse than a slow one, since nothing in a rendered image says how wrong it is. This section checks two properties of the Mandelbrot set that do not depend on any implementation choice, only on the mathematics from Section 2.3, and closes the chapter by confirming that every optimization measured so far changed speed and nothing else, the correctness class Section 6.4.5 sets apart from the rest of this chapter.

Figure 7.8.1’s left panel fits a box counting dimension to the rendered boundary, $D \approx 1.10$ with $R^2 = 0.9907$, well below the boundary’s known Hausdorff dimension of exactly 2, the value Section 2.3 gives, the expected gap since the asymptotic value needs boundary detail at scales finer than any finite render can reach. The right panel sweeps `max_iter` against the Mandelbrot set’s literature area estimate, roughly 1.50659177, and shows the error shrinking monotonically as the cap grows, exactly as the escape time approximation in Section 2.4 predicts, since a cap that is too small misclassifies slow escaping boundary pixels as interior and inflates the estimate. Neither result depends on any optimization in Chapter 3, both come from the plain scalar kernel, and both give the qualitative answer a differently implemented renderer would, evidence that the optimizations elsewhere in this thesis changed speed and not correctness.

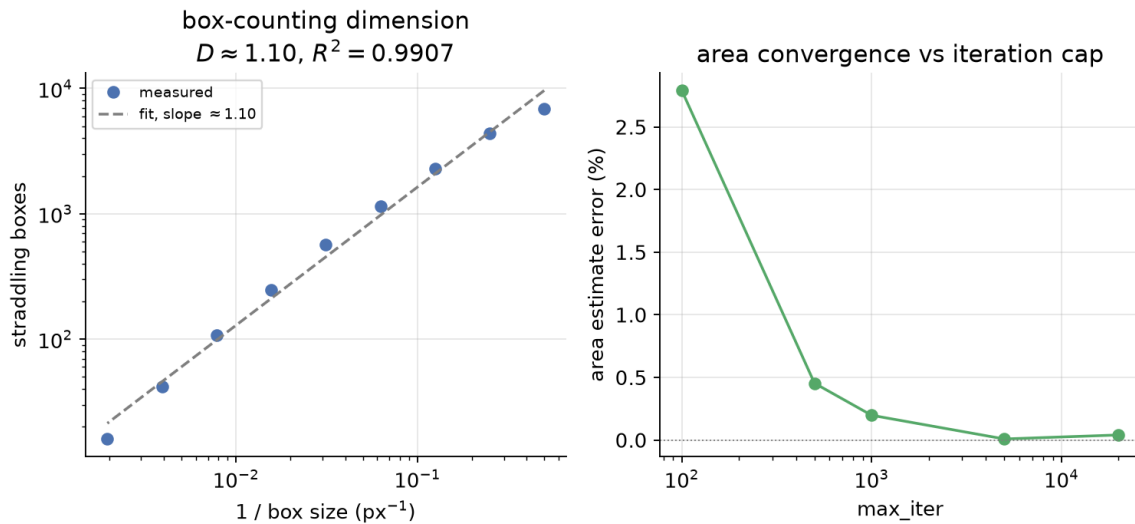


Figure 7.8.1: Numerical validation, box counting dimension of the rendered boundary and area estimate convergence against the iteration cap.

Both checks land where the mathematics says they should, independent of any speed result in the rest of this chapter. What the preceding sections measured as throughput, then, was always throughput on the correct answer.

8. CONCLUSION

This thesis set out to answer a question that most fractal renderers never actually ask. A typical personal computer has both a multicore CPU and a GPU sitting idle for most of a render, so why settle for one? The goal was to build a renderer that pushes a CPU path and a GPU path each as far as they individually go, and then to tie the two together with a scheduler that decides at runtime, tile by tile, which device should render what, rather than assuming the split in advance. The expectation going in was that the CPU path would need real optimization work before it was a fair partner for the GPU at all, that the GPU would win outright once that work was done, and that a scheduler would only earn its complexity where the two devices were closely enough matched for the choice of which one does what to actually matter. All three expectations held, though not always for the reasons assumed at the start, and the measurements in Chapter 7 are the record of where the guesses were right and where they were not.

The CPU side turned out to matter more than expected. A scalar baseline, carried through rayon parallel threading and then SIMD, reaches roughly a $1.9\times$ to $2.4\times$ speedup on its own depending on frame size, but the bigger win came from a cheaper idea, testing whether a pixel is provably inside the set before iterating it at all. That early exit test, plus catching cycles the naive loop would otherwise burn iterations on, is what let a single CPU thread outrun a comparable C++ renderer by a factor of roughly sixteen, with neither side touching a GPU. Not everything paid off. Hilbert curve traversal, chosen for its cache locality properties, was slower than a plain row major scan at every resolution tested, and perturbation theory, built for deep zoom, never won a single benchmark within the zoom range this thesis actually measured, because plain double precision arithmetic simply never broke down in that range.

The GPU delivered on its promise of raw throughput, reaching roughly 1389 million pixels a second at 4K, well ahead of every CPU configuration. What was not expected going in was where the GPU actually spends its time. The two GPU backends' compute kernels are within five percent of each other, and almost the entire gap between them comes from copying the finished image back to the CPU, not from computing it. Coloring the image afterward mattered more than the GPU versus GPU comparison itself. Measured with `colorize` still running on the CPU for both backends, that one step alone cut the GPU's advantage over the CPU to well under half of its render only lead. Moving it onto the GPU turned out to be the single largest performance gain this thesis identified, and for CUDA the thesis went on to implement it, an on device histogram and palette lookup kernel, pixel exact against the CPU version, 54% to 63% faster end to end at the shallow and mid zooms where `colorize` used to dominate frame time. `wgpu` and the heterogeneous scheduler still `colorize` on the CPU, the scheduler by necessity since its merged buffer only ever exists on the host side, so that gain remains a clearer direction for future work on those two paths.

The hybrid scheduler is the part of this thesis with the most story to tell. At a shallow zoom, where the CPU and the faster GPU are not close in speed, every version of the scheduler, adaptive or not, loses to simply picking the faster device and using it alone. The scheduler earns its complexity at the one point where the two devices actually become comparable, the precision threshold where both fall back to slower, higher precision arithmetic and end up within about 10% of each other's speed. There, routing expensive tiles to the

CPU instead of splitting the frame evenly beats not only the better device alone but also the theoretical best a blind split could achieve, by a factor of 1.75. That result is the clearest evidence this thesis has for its central claim, a scheduler that looks at the work before deciding who does it is worth more than one that just assumes. The advantage shrinks at deeper zoom levels, leaving real headroom the scheduler does not yet recover.

Against four existing open source renderers, rustler's own kernel was roughly one order of magnitude faster than the closest competitor and two orders of magnitude faster than the slowest, and none of that margin came from Rust as a language, since the same early exit and cycle detection work behind the CPU numbers is what accounts for it. A GPU is not automatically the right answer, either. It is markedly more efficient per watt than the CPU here, but the CPU still wins on raw throughput, so "faster" and "cheaper to run" point at different devices depending on what is actually being optimized for. And underneath all of the speed numbers sits a quieter but necessary result. A box counting estimate of the rendered boundary and a convergence check against the set's known area both land where the mathematics predicts they should, which is the evidence that a dozen rewrites of the same hot loop changed how fast this renderer is and nothing about whether it is correct.

Two directions follow directly from what this thesis measured rather than from speculation. Moving image coloring onto the GPU is the highest value change identified but not built, and tuning the scheduler's deep zoom behavior would close a real, already quantified gap rather than requiring a new idea. A third, smaller direction is honest bookkeeping. Perturbation theory was never tested at the zoom depths it was actually designed for, so its real payoff, as opposed to its measured absence of one, is still an open question. None of the three require rethinking anything in this thesis, only continuing it.

BIBLIOGRAPHY

- [1] Michael F. Barnsley. *Fractals Everywhere*. Academic Press, 1988.
- [2] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT Numerical Mathematics*, 20(2):176–184, 1980.
- [3] Maxime Collet. Fractals-rs: a Rust fractal explorer. <https://github.com/Maxime-Cltt/Fractals-rs>, 2026.
- [4] T. J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18(3):224–242, 1971.
- [5] Kenneth Falconer. *Fractal Geometry: Mathematical Foundations and Applications*. John Wiley & Sons, 3rd edition, 2014.
- [6] flutomax. Nanobrot: a fork of kalles fraktaler for deep-zoom mandelbrot rendering. <https://github.com/flutomax/nanobrot>, 2020.
- [7] Claude Heiland-Allen. Deep zoom theory and practice. https://mathr.co.uk/blog/2021-05-14_deep_zoom_theory_and_practice.html, 2021.
- [8] David Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38(3):459–460, 1891.
- [9] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.
- [10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In *Proceedings of the 37th International Symposium on Computer Architecture (ISCA)*, 2010.
- [11] Chi-Keung Luk, Sunpyo Hong, and Hyesoon Kim. Qilin: Exploiting parallelism on heterogeneous multiprocessors with adaptive mapping. In *Proceedings of the 42nd IEEE/ACM International Symposium on Microarchitecture (MICRO-42)*, 2009.
- [12] Benoit B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman, 1982.
- [13] K. I. Martin. Superfractalthing maths: Theory and practice of perturbation in complex dynamics. https://superfractalthing.co.nf/sft_maths.pdf, 2013.
- [14] Michał Męciński. Fraqtive: a SIMD and multi-core mandelbrot-family fractal generator. <https://github.com/mimecorg/fraqtive>, 2023.
- [15] John Milnor. *Dynamics in One Complex Variable*. Annals of Mathematics Studies. Princeton University Press, 3rd edition, 2006.

- [16] Guy M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. Technical report, IBM Ltd., Ottawa, Canada, 1966.
- [17] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. Accurate sum and dot product. *SIAM Journal on Scientific Computing*, 26(6):1955–1988, 2005.
- [18] Marc Pégon. Fractalnow: an open-source, multi-threaded fractal generator. <http://fractalnow.sourceforge.net/>, 2017.
- [19] Heinz-Otto Peitgen and Dietmar Saupe, editors. *The Science of Fractal Images*. Springer-Verlag, New York, 1988.
- [20] Mark Seufert. Fractalrenderercpp: a C++ fractal rendering library. <https://github.com/MarkSeufert/FractalRendererCpp>, 2021.
- [21] XaoS Project. XaoS: a real-time interactive fractal zoomer. <https://github.com/xaos-project/XaoS>, 2026.

BIOGRAPHY

Name Surname was born on dd.mm.yyyy in City. They completed primary school "...” and secondary school "...”. After finishing secondary school, they enrolled in the ... study programme at the Faculty of Technical Sciences in Novi Sad, where they studied from 20XX to 20XX.



КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Тип документације, ТД:	монографска публикација
Тип записа, ТЗ:	текстуални штампани документ
Врста рада, ВР:	Bachelor Thesis
Аутор, АУ:	Papp Tamás
Ментор, МН:	Dr Velko Petrović
Наслов рада, НР:	Fraktal renderovanje sa hibridnom paralelizacijom
Језик публикације, ЈП:	english
Језик извода, ЈИ:	српски / енглески
Земља публикавања, ЗП:	Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	ауторски репринт
Место и адреса, МА:	Нови Сад, Факултет техничких наука, Трг Доситеја Обрадовића 6
Физички опис рада, ФО:	бр. поглавља 8 / страница XX / цитата X / табела X / слика X / графика X / прилога X
Научна област, НО:	Софтверско инжењерство и информационе технологије
Научна дисциплина, НД:	Софтверско инжењерство
Предметна одредница / кључне речи, ПО:	keyword 1, keyword 2, keyword 3
УДК:	
Чува се, ЧУ:	Библиотека Факултета техничких наука, Трг Доситеја Обрадовића 6, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	One paragraph that captures the essence of the thesis: the problem, the motivation, an outline of the solution and the result.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	
Чланови комисије, КО:	
председник:	Dr Name Surname, title
члан:	Dr Name Surname, title
члан, ментор:	Dr Velko Petrović



KEY WORDS DOCUMENTATION

Document type, DT:	monographic publication
Type of record, TR:	textual material
Contents code, CC:	Bachelor Thesis
Author, AU:	Papp Tamás
Mentor, MN:	Dr Velko Petrović, PhD
Title, TI:	Fractal rendering with hybrid parallelism
Language of text, LT:	English
Language of abstract, LA:	Serbian / English
Country of publication, CP:	Serbia
Locality of publication, LP:	Vojvodina
Publication year, PY:	2026
Publisher, PB:	author's reprint
Publication place, PP:	Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6
Physical description, PD:	chapters 8 / pages XX / references X / tables X / illustrations X / graphs X / appendixes X
Scientific field, SF:	Software Engineering and Information Technologies
Scientific discipline, SD:	Software Engineering
Subject/Keywords, S/KW:	fractal, high performance computing, hybrid systems
UC:	
Holding data, HD:	Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad
Note, N:	
Abstract, AB:	One paragraph that captures the essence of the thesis: the problem, the motivation, an outline of the solution and the result.
Accepted by sci. Board on, ASB:	
Defended on, DE:	
Defense board, DB:	
president:	Name Surname, full professor, PhD
member:	Name Surname, full professor, PhD
member, mentor:	Dr Velko Petrović, PhD